VISVESVARAYA TECHNOLOGICAL UNIVERSITY
BELAGAVI – 590018



A PROJECT REPORT (PHASE-2) ON

INSIGHTLM: A LANGCHAIN-POWERED DOCUMENT RESEARCH SYSTEM

Submitted in partial fulfillment of the requirements for the award of degree of
BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE & ENGINEERING (DATA SCIENCE)

Submitted by:

- **DEEPAK S J** USN: 1MJ24CD402
- **M JAYANTH** USN: 1MJ24CD404
- **VARSHITHA H** USN: 1MJ24CD411
- **VENUGOPAL REDDY.C** USN: 1MJ24CD412

Under the Guidance of
Ms. Victoria Angelina
Assistant Professor, Department of CSE (Data Science)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (DATA SCIENCE)
MVJ COLLEGE OF ENGINEERING
(An Autonomous Institution, Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi & Accredited by NBA & NAAC)
Near ITPB, Channasandra, Kadugodi Post, Bengaluru – 560067
2025–2026

MVJ COLLEGE OF ENGINEERING
Engineering A Better Tomorrow
An Autonomous Institute
(Affiliated to Visvesvaraya Technological University, Belagavi
Approved By AICTE, New Delhi,
Recognized by UGC under 2(f) & 12(B)
Accredited by NBA and NAAC)

MVJ COLLEGE OF ENGINEERING

(An Autonomous Institution, Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi)
Accredited by NBA & NAAC, Near ITPB, Whitefield, Bengaluru – 560067
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (DATA SCIENCE)

CERTIFICATE

This is to certify that the Phase-2 project work entitled "INSIGHTLM: A LANGCHAIN-POWERED DOCUMENT RESEARCH SYSTEM" is a bonafide work carried out by DEEPAK S J (1MJ24CD402), M JAYANTH (1MJ24CD404), VARSHITHA H (1MJ24CD411), and VENUGOPAL REDDY.C (1MJ24CD412) in partial fulfillment for the award of Bachelor of Engineering in Computer Science & Engineering (Data Science) of Visvesvaraya Technological University, Belagavi during the academic year 2025–2026. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of Project Work prescribed for the said degree.

.....	.....	.....
Ms. Victoria Angelina	**Prof. B Sivasakthi**	**Dr. P. Mahabaleswarappa**
Project Guide	**Head of Department**	**Principal**
Assistant Professor, CSE (DS)	**Assoc. Professor & HOD, CSE (DS)**	**MVJCE, Bengaluru**

University Viva-Voce Examination:

Name of the Examiners	**Signature with Date**
1.	.....
2.	.....

DECLARATION

We, the undersigned students of the Department of Computer Science & Engineering (Data Science), MVJ College of Engineering, Bengaluru, hereby declare that the project work entitled "INSIGHTLM: A LANGCHAIN-POWERED DOCUMENT RESEARCH SYSTEM" has been independently carried out by us under the supervision of Ms. Victoria Angelina, Assistant Professor, Department of Computer Science & Engineering (Data Science), MVJCE, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science & Engineering (Data Science) from Visvesvaraya Technological University (VTU), Belagavi during the academic year 2025–2026.

We further declare that the matter embodied in this project report has not been submitted previously to any other university or institute for the award of any degree or diploma. All sources of literature, open-source libraries, academic papers, and frameworks referenced during the design, development, and testing of this software system have been duly cited and acknowledged in the bibliography.

Sl. No.	Name of the Student & USN	Signature
1	**DEEPAK S J (1MJ24CD402)**	.....
2	**M JAYANTH (1MJ24CD404)**	.....
3	**VARSHITHA H (1MJ24CD411)**	.....
4	**VENUGOPAL REDDY.C (1MJ24CD412)**	.....

Place: Bengaluru
Date: May 2026

ACKNOWLEDGEMENT

The successful completion of this Phase-2 project dissertation entitled "INSIGHTLM: A LANGCHAIN-POWERED DOCUMENT RESEARCH SYSTEM" has been a deeply enriching academic experience. We take this opportunity to express our profound gratitude and heartfelt appreciation to all individuals and authorities who supported, guided, and encouraged us throughout this endeavor.

First and foremost, we express our profound gratitude to our esteemed Project Guide, Ms. Victoria Angelina, Assistant Professor, Department of Computer Science & Engineering (Data Science), for her invaluable mentorship, constructive criticisms, intellectual guidance, and continuous encouragement during every stage of design, system architecture, RAG optimization, and rigorous testing.

We extend our sincere and heartfelt thanks to Prof. B Sivasakthi, Associate Professor & Head, Department of Computer Science & Engineering (Data Science), for providing necessary computational resources, laboratory facilities, academic advice, and institutional support throughout the development of this capstone project.

We express our deep sense of gratitude to Dr. P. Mahabaleswarappa, Principal, MVJ College of Engineering, for fostering an academic environment dedicated to technical innovation, research excellence, and hands-on software development.

We also convey our appreciation to the Project Coordinators, faculty members, and non-teaching staff of the Department of Computer Science & Engineering (Data Science) for their continuous administrative assistance and valuable suggestions during periodic internal project reviews.

Finally, we express our deepest indebtedness to our parents and family members for their enduring moral support and sacrifices, and to our peers whose collaborative discussions and constructive feedback helped shape this research system into a production-grade academic reality.

DEEPAK S J (1MJ24CD402)
M JAYANTH (1MJ24CD404)
VARSHITHA H (1MJ24CD411)
VENUGOPAL REDDY.C (1MJ24CD412)

ABSTRACT

The contemporary academic and scientific landscape is characterized by an exponential influx of published literature, textbooks, and multi-modal digital resources. Researchers, university faculty, and undergraduate students confront severe cognitive overload, information fragmentation, and prohibitive search latencies when synthesizing insights across extensive document corpora. While Large Language Models (LLMs) offer fluent natural language interaction, standalone foundational models suffer from debilitating vulnerabilities including factual hallucinations, lack of verifiable source attribution, temporal knowledge cutoffs, and context window saturation. To decisively overcome these foundational limitations, this capstone project presents INSIGHTLM: A LangChain-Powered Document Research System, developed in Phase 2 as a production-grade, full-stack, retrieval-augmented intelligence and pedagogical assessment platform.

The system architecture employs a decoupled client-server paradigm. The frontend is engineered with React 18, TypeScript, Tailwind CSS, Lucide Vector Icons, and Vite, delivering an accessible, role-customized user experience across diverse academic workflows. The backend application server is built on Python 3.11 and Flask 3.1, incorporating a high-performance RESTful API gateway, CORS reverse-proxying, and cryptographic session management. Core document intelligence is driven by a LangChain Retrieval-Augmented Generation (RAG) orchestration pipeline. Ingested Portable Document Format (PDF) files undergo structural extraction via PyPDF, followed by semantic chunking through a RecursiveCharacterTextSplitter configured with a 1,000-character window and 200-character sliding overlap. Chunk representations are vectorized into 384-dimensional dense semantic embeddings using the all-MiniLM-L6-v2 transformer and indexed inside a persistent ChromaDB v0.5 vector database using Hierarchical Navigable Small World (HNSW) graph indexing with cosine similarity distance metric.

A cornerstone innovation of the Phase 2 implementation is the integration of an enterprise-grade Role-Based Access Control (RBAC) security model featuring salted Scrypt cryptographic password hashing. The system ships with six authentic Indian academic persona accounts across three specialized roles: Students (Aarav Sharma and Priya Patel), Teachers (Prof. Rahul Verma and Prof. Sneha Iyer), and Researchers (Dr. Arjun Reddy and Dr. Ananya Nair). Each persona accesses a dedicated, role-customized

dashboard tailored to their respective workflow. For educators, INSIGHTLM introduces a pedagogical Question Paper Studio capable of deterministically generating examination questions across eight distinct formats: Multiple Choice Questions (MCQ), Fill in the Blanks, True/False, Short Answer, Numerical Problems, Assertion-Reason, Match the Following, and Long Essay questions, tagged according to Bloom's Revised Cognitive Taxonomy. Students interact with timed examination modules supported by an automated grading engine that computes instantaneous objective scores and detailed rubric feedback.

Furthermore, INSIGHTLM integrates cross-corpus multi-modal learning capabilities, including contextual YouTube lecture discovery via YouTube Data API v3, synthesized audio voiceover generation through the browser-native Web Speech API, 10-dimensional cross-paper comparative matrix generation, and a standardized scientific citation generator producing formatted references in IEEE, APA, MLA, and BibTeX standards. Comprehensive quality assurance testing executed against the live platform yielded a 100% pass rate across 71 automated test cases, confirming sub-500ms vector retrieval latencies, absolute cryptographic isolation, and zero plaintext credential leakage. INSIGHTLM establishes a reproducible, scalable benchmark for next-generation AI-assisted academic research and learning environments.

Keywords: *Retrieval-Augmented Generation (RAG), LangChain, ChromaDB, HNSW Vector Indexing, Role-Based Access Control (RBAC), Scrypt Cryptography, Bloom's Taxonomy, Automated Assessment, Multi-Modal Learning, React TypeScript.*

TABLE OF CONTENTS

Certificate ii
Declaration iii
Acknowledgement iv
Abstract v
Table of Contents vii
List of Figures ix
List of Tables x
List of Abbreviations xii
CHAPTER 1: INTRODUCTION **13**
1.1 Background and Context 13
1.2 Motivation 14
1.3 Project Overview & Scope 15
1.4 Objectives of Phase 2 Project 16
1.5 Organization of the Report 17
CHAPTER 2: LITERATURE SURVEY **19**
2.1 Survey of Existing Document Analysis & RAG Systems 19
2.2 Critical Analysis & Research Gaps 21
2.3 Summary Table of Literature Survey 22
CHAPTER 3: PROBLEM STATEMENT & PROPOSED SYSTEM **24**
3.1 Problem Statement 24
3.2 Limitations of the Existing System 24
3.3 Proposed System Architecture & Innovations 25
3.4 Key Features and Value Propositions 26
CHAPTER 4: REQUIREMENTS SPECIFICATION **27**
4.1 Functional Requirements 27
4.2 Non-Functional Requirements 28
4.3 Hardware Requirements 29
4.4 Software Requirements 30
CHAPTER 5: SYSTEM DESIGN & ARCHITECTURE **32**
5.1 Overall Architectural Design (6-Layer Architecture) 32
5.2 System Workflow Pipeline 33
5.3 Role-Based Access Control (RBAC) & Use Case Modeling 34
5.4 Sequence & Interaction Modeling 35
5.5 Data Flow Modeling (Level 0 and Level 1 DFDs) 36
5.6 Database & Vector Store Schema Design 38
CHAPTER 6: IMPLEMENTATION & **41**

METHODOLOGY
6.1 Frontend Engineering (React 18 + TypeScript + Vite) 41
6.2 Backend Services & REST API Gateway (Flask 3.1) 42
6.3 Retrieval-Augmented Generation (RAG) Implementation 43
6.4 Cryptographic Security & Indian Account Implementation 44
6.5 Pedagogical Question Paper Studio (8 Formats) 44
6.6 Automated Assessment & Instant Grading Engine 45
6.7 Multi-Modal Research Integration (YouTube & Speech TTS) 46
6.8 Scientific Citation Engine (IEEE, APA, MLA, BibTeX) 46
CHAPTER 7: RESULTS AND DISCUSSION **47**
7.1 User Interface Implementation & Verification 47
7.2 Experimental Performance Evaluation 58
7.3 Comparative Analysis with Baseline Systems 59
7.4 Discussion on Findings and Usability 60
CHAPTER 8: SYSTEM TESTING AND VALIDATION **61**
8.1 QA Testing Methodology and Automation Framework 61
8.2 Execution Results of the 71 Automated Test Cases 61
8.3 Security & Role-Based Access Control Verification 65
8.4 Cross-Browser and Stress Testing 65
CHAPTER 9: LIMITATIONS AND FUTURE ENHANCEMENTS **66**
9.1 Technical Limitations 66
9.2 Future Enhancements 66
CHAPTER 10: CONCLUSION **68**
10.1 Concluding Remarks 68
10.2 Summary of Contributions 68
REFERENCES **70**

LIST OF FIGURES

Figure No.	Figure Caption Title	Page No.
Fig. 5.1	6-Layer System Architecture of INSIGHTLM	33
Fig. 5.2	End-to-End RAG Ingestion, Retrieval & Pedagogical Workflow	34
Fig. 5.3	Role-Based Access Control (RBAC) Use Case Diagram	35
Fig. 5.4	RAG Query, Vector Retrieval & Exam Submission Sequence Diagram	36
Fig. 5.5	Level 0 and Level 1 Data Flow Diagram (DFD)	37
Fig. 5.6	Relational & Vector Database Entity-Relationship (ER) Model	38
Fig. 7.1	Academic Access Portal with Scrypt Authentication & Indian Personas	48
Fig. 7.2	Student Learning Dashboard (Aarav Sharma)	49
Fig. 7.3	Document Management Repository & Indexed Vector Corpus	50
Fig. 7.4	Research Assistant with Grounded Q&A, Inline Citations & TTS Narration	51
Fig. 7.5	Student Assessment Studio & Timed Examination Interface	52
Fig. 7.6	YouTube Video Discovery & Topic-Specific Project Shelves	53
Fig. 7.7	Teacher Teaching Dashboard & Curriculum Metrics (Prof. Rahul Verma)	54
Fig. 7.8	Teacher Question Paper Studio with 8 Assessment Formats	55
Fig. 7.9	Student Exam Submission & Instant Auto-Grading Evaluation	56
Fig. 7.10	Researcher Academic Workspace & Analytics (Dr. Arjun Reddy)	57
Fig. 7.11	10-Dimensional Cross-Document Comparison Matrix	57
Fig. 7.12	Standardized Citation Generator (IEEE, APA, MLA, BibTeX)	58

LIST OF TABLES

Table No.	Table Title Description	Page No.
Table 2.1	Comparative Summary of Literature Survey Papers	22
Table 4.1	Functional Requirements Specification (FR-01 to FR-12)	27
Table 4.2	Non-Functional Requirements Specification (NFR-01 to NFR-08)	28
Table 4.3	Hardware & Computational Environment Specifications	29
Table 4.4	Software Stack & Technical Dependency Specifications	30
Table 5.1	USERS Relational Schema and Cryptographic Attributes	38
Table 5.2	DOCUMENTS Relational Schema and Metadata Attributes	39
Table 5.3	QUESTION_PAPERS & SUBMISSIONS Relational Schema	39
Table 5.4	CHROMA_CHUNKS High-Dimensional Vector Schema	40
Table 6.1	Supported 8 Question Formats Mapped to Bloom's Cognitive Levels	44
Table 7.1	System Latency & Vector Retrieval Benchmark Results	58
Table 7.2	Comparative Benchmark: INSIGHTLM vs. Alternative Architectures	59
Table 8.1	Test Suite 1: Authentication & Indian Persona Accounts (14 Tests)	61
Table 8.2	Test Suite 2: Roles, Assessments & Question Studio (7 Tests)	62
Table 8.3	Test Suite 3: RAG Query Archetypes & Accuracy Validation (16 Tests)	62
Table 8.4	Test Suite 4: Security, RBAC & Performance Benchmarks (6 Tests)	63
Table 8.8	Overall QA Automated Testing Suite Execution Metrics	64

LIST OF ABBREVIATIONS

Abbreviation	Expansion / Full Term
AI	Artificial Intelligence
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSE	Computer Science & Engineering
DFD	Data Flow Diagram
DPR	Dense Passage Retrieval
ERD	Entity-Relationship Diagram
HNSW	Hierarchical Navigable Small World (Graph Index)
HTTP	Hypertext Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
LLM	Large Language Model
MCQ	Multiple Choice Question
MVJCE	MVJ College of Engineering
NFR	Non-Functional Requirement
NLP	Natural Language Processing
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
SBERT	Sentence-BERT (Bidirectional Encoder Representations from Transformers)
Scrypt	Memory-Hard Cryptographic Key Derivation Function
TF-IDF	Term Frequency - Inverse Document Frequency
TS	TypeScript
TTS	Text-to-Speech
UI/UX	User Interface / User Experience
USN	University Seat Number
VTU	Visvesvaraya Technological University
WSGI	Web Server Gateway Interface

CHAPTER 1
INTRODUCTION

1.1 Background and Context

In modern higher education, scientific research, and professional engineering, digital literature has grown at an unprecedented rate. Millions of peer-reviewed journal papers, academic textbooks, institutional dissertations, and technical specifications are published each year. While this wealth of information presents profound opportunities for technological discovery, it has concurrently generated an acute information overload. Engineering students, academic faculty, and scientific researchers spend an inordinate proportion of their working hours manually scanning dense, multi-page Portable Document Format (PDF) files to locate specific formulas, methodologies, algorithmic proofs, and empirical findings. Traditional keyword-based retrieval utilities (such as standard desktop PDF search or relational database string lookups) rely strictly on exact lexical matching, rendering them utterly blind to semantic equivalence, conceptual synonyms, and contextual nuances.

The recent emergence of foundational Large Language Models (LLMs), powered by multi-billion parameter transformer architectures, has fundamentally redefined human interaction with textual knowledge. Models such as GPT-4, Gemini, and LLaMA demonstrate extraordinary capabilities in natural language reasoning, abstractive summarization, code generation, and multi-turn conversational dialogue. However, despite their impressive linguistic fluency, foundational LLMs suffer from four debilitating structural deficiencies when applied directly to serious academic and technical research without architectural augmentation:

1. Hallucinatory Vulnerabilities: Foundational generative models operate probabilistically, predicting next tokens based on statistical associations rather than verifiable factual grounding. When queried regarding esoteric mathematical equations, niche scientific papers, or specific departmental syllabi, standalone LLMs frequently fabricate plausible-sounding yet completely erroneous citations, figures, and conclusions.

2. Absence of Direct Citation Attribution: Traditional autoregressive language models do not intrinsically furnish verifiable page-level or chunk-level attribution. In rigorous collegiate academic environments, an ungrounded assertion is scientifically unacceptable;

every claim, numerical benchmark, and definition must be traceable to a specific paragraph, table, or page within an authoritative primary source.

3. Knowledge Latency and Cutoff Boundaries: Foundational models are bounded by their static pre-training datasets. They possess zero native awareness of proprietary institutional documents, recently uploaded research preprints, or private course textbooks published after their training cutoff date.

4. Context Window Saturation and Economic Costs: While recent models support extended context windows (e.g., 32k or 128k tokens), stuffing an entire 500-page engineering textbook or dozens of research papers into a single context prompt incurs prohibitive inference latency, quadratic computational attention costs, and the well-documented 'lost-in-the-middle' phenomenon where LLMs fail to retrieve critical details embedded in lengthy contexts.

To systematically address these bottlenecks, the paradigm of Retrieval-Augmented Generation (RAG) has emerged as the definitive state-of-the-art framework. In a RAG architecture, an external, non-parametric knowledge corpus is segmented, vectorized into a high-dimensional continuous semantic space, and indexed inside an optimized vector database. When a user submits an inquiry, the system first retrieves the top-k most semantically relevant text passages via dense vector similarity matching and subsequently injects these retrieved passages as factual grounding context into the prompt passed to the generative language model.

1.2 Motivation

The motivation driving the development of INSIGHTLM: A LangChain-Powered Document Research System stems directly from observed pedagogical and research inefficiencies within academic institutions such as MVJ College of Engineering. Specifically:

• Student Learning Inefficiencies: Undergraduate engineering students frequently struggle to extract core conceptual intuitions from dense scientific literature. When preparing for university examinations, they lack unified tools to quickly synthesize textbook definitions, obtain instant clarification on complex diagrams, and test their comprehension against model examination questions.

• Faculty Assessment Overhead: Academic professors and course coordinators invest substantial time manually drafting balanced examination papers across different cognitive

levels (Bloom's Taxonomy). Generating diverse question formats—ranging from Multiple Choice Questions (MCQs) to numerical problems, assertion-reason items, and long analytical essays—demands extensive manual effort.

• Researcher Literature Review Bottlenecks: Research scholars embarking on literature surveys must manually read across multiple competing papers to construct comparative methodology matrices, identify unaddressed research gaps, and format citations according to IEEE, APA, or BibTeX conventions.

• Lack of Multi-Modal Integration: While students increasingly rely on video lectures (e.g., NPTEL, MIT OpenCourseWare, YouTube) to supplement textbook reading, existing document systems operate in total isolation from external video repositories, failing to connect textual topics with relevant visual lectures.

INSIGHTLM was conceived to synthesize these disparate requirements into a single, cohesive, production-grade platform that unifies document ingestion, dense vector retrieval, role-based pedagogical intelligence, automated examination synthesis, and multi-modal lecture discovery.

1.3 Project Overview & Scope

INSIGHTLM is an end-to-end, full-stack, enterprise-grade AI research and learning platform engineered specifically for academic ecosystems. Developing upon the foundational concepts explored during Phase 1, the Phase 2 implementation transforms the preliminary concept into a fully realized, decoupled, and production-tested software platform.

The scope of INSIGHTLM encompasses the following primary functional domains:

1. Document Ingestion and Vector Indexing: Automated parsing of multi-page PDF documents, semantic sliding-window chunking (1,000 characters with 200-character overlap), 384-dimensional dense semantic embedding generation using all-MiniLM-L6-v2, and persistent HNSW graph indexing inside ChromaDB v0.5.

2. Grounded Multi-Turn Question Answering: Interactive conversational assistant providing factual responses strictly grounded in indexed document chunks, complete with page-level citations, similarity confidence scores, and honest admission of out-of-scope queries.

3. Role-Based Access Control (RBAC): Cryptographically secured multi-tenant access utilizing salted Scrypt hashing, featuring pre-configured Indian academic persona accounts for Students, Teachers, and Researchers.

4. Teacher Pedagogical Assessment Studio: Automated generation of university model question papers spanning eight standardized formats, tagged across Bloom's Revised Cognitive Taxonomy levels, with automated mark allocation.

5. Automated Student Examination & Grading: Interactive timed test-taking interface featuring live countdown timers, instantaneous deterministic auto-scoring for objective questions, and structured rubric-based feedback.

6. Multi-Perspective Document Synthesis: Automated generation of six structured summary types (Executive, Pedagogical, Methodological, Chronological, Technical, and Critical Analysis) and 10-dimensional cross-paper comparative matrices.

7. Multi-Modal Learning & YouTube Discovery: Contextual matching of indexed research topics with high-quality academic YouTube video lectures via YouTube Data API v3 and speech voiceover narration via browser Web Speech API.

8. Standardized Scientific Citation Engine: One-click serialization of indexed papers and synthesis summaries into IEEE, APA, MLA, and BibTeX citation standards.

1.4 Objectives of Phase 2 Project

The Phase 2 development of INSIGHTLM was guided by the following concrete, measurable technical objectives:

• Primary Objective 1: Implement a high-performance, decoupled client-server architecture combining a responsive React 18 / TypeScript / Tailwind CSS frontend with a robust Python 3.11 / Flask 3.1 REST API gateway.

• Primary Objective 2: Engineer an optimized LangChain RAG pipeline with ChromaDB v0.5 persistent vector storage, achieving sub-500ms top-k cosine similarity retrieval latencies across multi-document corpora.

• Primary Objective 3: Establish an enterprise-grade RBAC security model backed by salted Scrypt password hashing, pre-configuring six Indian academic personas across Student, Teacher, and Researcher roles with complete privilege isolation.

• Primary Objective 4: Design and deploy a Teacher Assessment Studio capable of synthesizing examination papers across eight standardized question formats with automatic Bloom's taxonomy mapping and auto-grading capabilities.

• Primary Objective 5: Integrate multi-modal educational features, specifically automated YouTube lecture discovery via YouTube Data API v3, browser-native Text-to-Speech narration, and a multi-standard citation generator.

• Primary Objective 6: Execute comprehensive automated QA verification across unit, integration, security, and end-to-end workflows, achieving 100% test pass rates across all functional modules.

1.5 Organization of the Report

The remainder of this dissertation is organized as follows:

• Chapter 2 (Literature Survey) presents an exhaustive review of seminal literature in dense retrieval, RAG architectures, vector indexing, and automated pedagogical evaluation, concluding with a comparative analysis of related systems.

• Chapter 3 (Problem Statement & Proposed System) formally specifies the technical research problem, analyzes the limitations of existing solutions, and articulates the innovations introduced by INSIGHTLM.

• Chapter 4 (Requirements Specification) provides detailed functional requirements (FR-01 to FR-12), non-functional constraints (NFR-01 to NFR-08), and hardware/software specifications.

• Chapter 5 (System Design & Architecture) details the 6-layer architecture, end-to-end workflows, RBAC use cases, sequence interactions, data flow diagrams (Level 0 and Level 1), and relational/vector database schemas.

• Chapter 6 (Implementation & Methodology) offers an in-depth technical walkthrough of the frontend engineering, Flask backend gateway, RAG pipeline, Scrypt authentication, 8-format question generation, auto-grading, and multi-modal integrations.

• Chapter 7 (Results and Discussion) showcases the live system implementation through 12 verified screenshots, quantitative latency benchmarks, comparative architectural analyses, and academic usability discussions.

• Chapter 8 (System Testing and Validation) documents the execution of 71 automated tests across seven distinct test suites, accompanied by security verification and performance evaluations.

• Chapter 9 (Limitations and Future Enhancements) discusses current technical constraints and identifies concrete directions for future research.

• Chapter 10 (Conclusion) summarizes the primary contributions and concludes the dissertation.

CHAPTER 2
LITERATURE SURVEY

2.1 Survey of Existing Document Analysis & RAG Systems

The development of intelligent document research assistants represents the convergence of three foundational fields in Computer Science: Information Retrieval (IR), Natural Language Processing (NLP), and Dense Semantic Vector Indexing. Over the past five years, the transition from sparse term-frequency representations to dense neural representations has radically reshaped how software systems index and retrieve unstructured academic text.

2.1.1 Evolution from Lexical Search to Dense Semantic Retrieval

Traditional information retrieval systems relied overwhelmingly on inverted index data structures and probabilistic term-weighting algorithms, most notably BM25 (Robertson et al., 2009) and TF-IDF (Salton & Buckley, 1988). While BM25 provides exceptional computational efficiency and exact lexical precision, it suffers from severe vocabulary mismatch: if a user queries 'convolutional feature map downsampling', BM25 cannot retrieve relevant passages that describe 'spatial pooling layers' unless both terms explicitly co-occur. In dense academic research literature, where complex theoretical ideas are routinely expressed using varied terminology, lexical systems exhibit unacceptably high false-negative retrieval rates.

The introduction of Transformer-based contextual language models (Vaswani et al., 2017; Devlin et al., 2019) enabled the encoding of entire sentences and paragraphs into continuous, high-dimensional vector spaces. Reimers and Gurevych (2019) introduced Sentence-BERT (SBERT), utilizing Siamese and triplet network structures to generate semantically meaningful sentence embeddings that can be compared using cosine similarity. SBERT reduced the computational cost of finding the most similar pair of sentences from 65 hours (with cross-encoder BERT) to approximately 5 seconds, laying the empirical foundation for modern dense passage retrieval (DPR) pipelines (Karpukhin et al., 2020).

2.1.2 Foundations of Retrieval-Augmented Generation (RAG)

In their seminal paper, Lewis et al. (2020) formalized the Retrieval-Augmented Generation (RAG) architecture. Rather than relying entirely on parametric memory (the

static weights of a pre-trained neural network), RAG pairs a parametric sequence-to-sequence model (such as BART or T5) with a non-parametric retrieval component (a dense vector index of Wikipedia passages). When conditioned on an input prompt, the retriever fetches top-ranked passages, which are concatenated with the query and supplied to the generator. Lewis et al. demonstrated that RAG dramatically reduces hallucinations, provides superior factual precision on open-domain question answering benchmarks (Natural Questions, WebQuestions, CuratedTREC), and allows for effortless knowledge base updates simply by updating the external index without retraining the model.

Recent surveys by Gao et al. (2023) categorize RAG systems into three architectural generations: Naive RAG (basic chunk-embed-retrieve pipelines), Advanced RAG (incorporating pre-retrieval query rewriting, chunk reranking, and sliding-window segmentation), and Modular RAG (decoupled microservices supporting dynamic routing, multi-source orchestration, and iterative self-reflective retrieval). INSIGHTLM embodies an Advanced Modular RAG design, combining sliding-window chunking, deterministic relevance filtering, multi-modal video lecture enrichment, and role-based response adaptation.

2.1.3 High-Dimensional Vector Indexing and HNSW Graphs

A critical bottleneck in large-scale RAG systems is the nearest-neighbor search latency across millions of dense embedding vectors. Exact k-nearest-neighbor (k-NN) search requires computing the Euclidean or cosine distance against every vector in the corpus (an O(N * d) linear scan), which becomes computationally intractable for real-time interactive applications. Malkov and Yashunin (2018) resolved this challenge through the Hierarchical Navigable Small World (HNSW) graph algorithm. HNSW constructs a multi-layer graph where lower layers contain dense clusters of vectors and upper layers contain long-range 'skip' connections, analogous to skip-lists in one-dimensional data structures. HNSW achieves logarithmic search complexity (O(log N)) with state-of-the-art recall accuracy, making it the vector indexing algorithm of choice in modern embedded vector engines such as ChromaDB, Milvus, and Faiss.

2.1.4 Automated Question Generation and Pedagogical Bloom's Taxonomy

In educational technology, automated question generation (AQG) has evolved from rigid rule-based syntactic transformations (Heilman & Smith, 2010) to neural sequence-to-sequence generation (Du et al., 2017). However, purely generative question generators

often produce factually ambiguous questions or fail to adhere to standardized academic examination formats. Anderson and Krathwohl's (2001) Revised Bloom's Taxonomy categorizes cognitive educational objectives into six hierarchical levels: Remembering, Understanding, Applying, Analyzing, Evaluating, and Creating. Recent research by Wang et al. (2022) emphasizes that robust automated educational assessment requires structural format diversity (combining objective MCQs, fill-in-the-blanks, and numerical problems with subjective analytical essays) aligned explicitly with Bloom's cognitive levels. INSIGHTLM operationalizes this pedagogical theory by integrating eight distinct question formats with automated Bloom's taxonomy tagging.

2.2 Critical Analysis & Research Gaps

Despite significant commercial and academic interest in document research tools, existing platforms exhibit prominent structural and functional limitations when evaluated against university academic requirements:

1. Academic Role Blindness: Commercial systems (such as NotebookLM, ChatPDF, and Perplexity) provide a single, undifferentiated user interface. They fail to distinguish between the pedagogical needs of a teacher drafting a balanced examination, an undergraduate student revising concepts before a test, and a researcher synthesizing methodological tradeoffs across multiple literature preprints.

2. Monolithic Single-Document Focus: Many tools restrict conversational queries to a single uploaded document at a time. They cannot perform cross-corpus synthesis, multi-paper comparative matrix extraction, or holistic concept exploration across an entire department syllabus.

3. Inadequate Pedagogical Assessment Support: Existing tools lack automated question paper generation tailored to university examination formats. None of the mainstream commercial RAG assistants provide structured 8-format examination paper synthesis, automated student test submission workflows, or deterministic auto-grading engines.

4. Absence of Multi-Modal Media Enrichment: Academic learning is fundamentally multi-modal. Textual explanations frequently require video lecture demonstrations to solidify conceptual comprehension. Existing document systems operate in total disconnection from video platforms, requiring students to manually switch contexts to find related YouTube tutorials.

5. Cryptographic Insecurity in Academic Personas: Most open-source demonstration platforms store mock credentials in plaintext or employ simplistic session spoofing without robust, memory-hard cryptographic key derivation (such as Scrypt), rendering them vulnerable to trivial unauthorized role escalation.

2.3 Summary Table of Literature Survey

Table 2.1 presents a comprehensive comparative summary of seminal literature and commercial systems, highlighting their methodologies, key strengths, documented limitations, and direct relevance to the design of INSIGHTLM.

Table 2.1: Comparative Summary of Literature Survey Papers and Related Systems

Author & Year	Methodology / Architecture	Key Strengths	Documented Limitations	Relevance to INSIGHTLM
Lewis et al. (2020)	Retrieval-Augmented Generation (RAG) with BART & DPR	Pioneered non-parametric memory; reduced hallucinations	Evaluated on open Wikipedia; lack of multi-format document parsing	Core foundational RAG architecture for INSIGHTLM
Karpukhin et al. (2020)	Dense Passage Retrieval (DPR) with dual BERT encoders	Substantially outperformed BM25 lexical baselines in recall	Computationally intensive offline index generation	Informed dense embedding selection (MiniLM-L6-v2)
Reimers & Gurevych (2019)	Sentence-BERT (SBERT) Siamese Triplet Networks	Enabled fast, semantically meaningful sentence embeddings	Limited to 512 token context; requires text chunking	Directly utilized for semantic chunk representations
Malkov & Yashunin (2018)	Hierarchical Navigable Small World (HNSW) Graphs	Logarithmic vector search complexity O(log N); high recall	Memory consumption grows with index graph density	Core indexing engine in ChromaDB persistent store
Gao et al. (2023)	Modular RAG Survey & Taxonomy Analysis	Defined unified framework for pre-, during, and post-retrieval	Survey paper; no single end-to-end production artifact	Guided decoupled 6-layer architecture of INSIGHTLM
Anderson & Krathwohl (2001)	Revised Bloom's Cognitive Taxonomy Framework	Standardized pedagogical classification (Remember -> Create)	Theoretical educational model; lacks software automation	Implemented in Teacher Question Paper Studio
Wang et al. (2022)	Automated Assessment & Multi-Format	Demonstrated benefits of multi-format question	Limited format variety; lacked student auto-	Inspired INSIGHTLM's 8 question format

	Examination AQG	banks	grading pipeline	generator
Google NotebookLM (2023)	Proprietary Gemini RAG with Source Notes & Audio Overviews	Polished audio generation; strong multi-document grounding	Closed ecosystem; single generic persona; no teacher exam studio	Benchmark for citation grounding & user experience
ChatPDF (2023)	Single-document PDF chat interface using OpenAI API	Zero-configuration web utility for casual users	Single document only; no RBAC, exams, or multi-modal video	Illustrates baseline single-document limitations
Perplexity AI (2023)	Live Web RAG with inline numbered citations	Exceptional live web freshness and source attribution	Targeted at general search; lacks university course integration	Informed inline citation UI and source modal design

CHAPTER 3
PROBLEM STATEMENT & PROPOSED SYSTEM

3.1 Problem Statement

The central problem addressed by this project can be formally stated as follows:

"To design, engineer, validate, and deploy an intelligent, role-aware academic research and learning platform—INSIGHTLM—that leverages a decoupled client-server architecture, LangChain Retrieval-Augmented Generation, and persistent ChromaDB vector storage to eliminate factual hallucinations, provide verifiable page-level citation attribution across multi-document corpora, enforce cryptographically secured Role-Based Access Control across student, teacher, and researcher personas, automate pedagogical examination generation across eight standardized formats aligned with Bloom's Taxonomy, enable deterministic instant auto-grading, and integrate contextual multi-modal video lecture discovery."

3.2 Limitations of the Existing System

The Phase 1 preliminary prototype of the Intelligent Document Research Assistant provided valuable proof-of-concept validation for basic document chat. However, systematic architectural review and testing revealed several prominent limitations:

1. Prototype Monolithic Architecture: Phase 1 proposed a tightly coupled Streamlit-based interface that intermingled presentation, business logic, and database operations in a single execution thread, precluding responsive multi-tab interactions, modern client-side state caching, and mobile responsiveness.

2. Non-Persistent In-Memory Vectors: The initial prototype stored vector embeddings in transient in-memory collections. Consequently, whenever the server process restarted, all indexed documents, chunk boundaries, and vector representations were lost, necessitating complete re-parsing and re-embedding.

3. Absence of Role-Based Security: The Phase 1 prototype featured no authenticated security boundary. All users shared an identical unstructured view, with zero distinction between student learners, examining professors, and publishing researchers.

4. No Pedagogical Examination Infrastructure: Faculty members had no capability to generate curriculum-aligned question papers, define marking schemes, or conduct automated student assessments.

5. Single-Modal Textual Isolation: The system possessed zero connectivity with external video repositories or browser-native speech synthesis, restricting user interaction entirely to static text reading.

6. Ungrounded Hallucinatory Risk: When queried regarding concepts absent from uploaded documents, the preliminary prototype occasionally generated speculative answers without explicitly declaring knowledge boundary limitations.

3.3 Proposed System Architecture & Innovations

In Phase 2, INSIGHTLM has been completely re-architected into a modern, production-grade, decoupled client-server system that resolves every identified limitation through the following engineering innovations:

• Modern Decoupled Full-Stack Framework: A modern React 18, TypeScript, and Tailwind CSS frontend powered by Vite communicates asynchronously via a RESTful JSON API with a robust Python 3.11 / Flask 3.1 application server. This separation of concerns guarantees sub-second UI responsiveness, granular error boundary containment, and clean modular maintainability.

• Persistent ChromaDB Vector Store: Document embeddings are permanently stored in a persistent ChromaDB v0.5 collection backed by Parquet storage and HNSW graph indexing on disk (`backend/data/chroma_db`). System restarts preserve all vector representations without requiring re-computation.

• Cryptographic RBAC with Salted Scrypt: Complete multi-role authorization enforcing strict privilege boundaries between Students, Teachers, and Researchers. Passwords are never stored in plaintext, but rather hashed using salted, memory-hard Scrypt key derivation.

• 8-Format Pedagogical Question Paper Studio: Faculty members can instantly generate balanced examination papers containing Multiple Choice Questions, Fill in the Blanks, True/False, Short Answer, Numerical Problems, Assertion-Reason, Match the Following, and Long Analytical Essays, tagged with Bloom's cognitive taxonomy levels.

• Deterministic Auto-Grading Engine: Timed student exam submissions undergo instant automated evaluation for objective question items, computing overall percentages and presenting question-by-question explanations.

• Multi-Modal Lecture Integration: Context-aware YouTube lecture recommendations via YouTube Data API v3 and browser-native text-to-speech audio narration provide a rich, multi-modal educational experience.

• 10-Dimensional Cross-Paper Comparison Matrix: Researchers can synthesize methodological frameworks across up to five papers simultaneously, evaluating Problem Formulation, Core Methodology, Dataset, Empirical Metrics, Strengths, Limitations, and Research Gaps.

• Multi-Style Scientific Citation Generator: Automatic compilation of citations in IEEE, APA, MLA, and BibTeX standards for any uploaded paper or synthesized answer.

3.4 Key Features and Value Propositions

INSIGHTLM delivers distinct value propositions tailored to the three primary user segments of an academic engineering institution:

1. Value for Students: Transforms passive textbook reading into an active, conversational learning dialogue. Students can ask conceptual questions, review grounded citations, take timed practice tests with instant feedback, and watch curated YouTube lectures on challenging topics.

2. Value for Teachers: Automates the laborious task of drafting balanced examination papers and quizzes across eight standardized formats, provides complete control over question difficulty and marks, and streamlines the evaluation of student submissions.

3. Value for Researchers: Accelerates the literature review process by automating multi-document synthesis, cross-methodology comparisons, research gap discovery, and academic bibliography generation.

4. Value for the Institution: Provides a self-contained, privacy-preserving research intelligence platform that runs entirely within the university computational infrastructure without exposing proprietary curricula or student examination data to third-party data-scraping platforms.

CHAPTER 4
REQUIREMENTS SPECIFICATION

4.1 Functional Requirements

Functional requirements define the core capabilities, operations, and services that INSIGHTLM must provide to its end users. Table 4.1 specifies the twelve primary functional requirements (FR-01 to FR-12) implemented and verified in the Phase 2 system.

Table 4.1: Functional Requirements Specification (FR-01 to FR-12)

Req ID	Requirement Name	Detailed Functional Description	Target Persona / Role	Verification Status
FR-01	User Authentication & Scrypt Cryptography	Platform must authenticate users via unique username/email and password verified against salted Scrypt hashes in users.json; enforce password length >6 chars and reject invalid logins with HTTP 401.	All Roles	Verified (Suite 1)
FR-02	Role-Based Access Control (RBAC)	System must isolate feature sets based on user role (Student, Teacher, Researcher); enforce strict 403 Forbidden blocks when unauthorized roles attempt privileged operations.	All Roles	Verified (Suite 1, 2)
FR-03	Multi-Page PDF Ingestion & Validation	Platform must accept PDF file uploads, validate file headers, parse raw text page-by-page via PyPDF, normalize whitespace, and reject empty, corrupted, or non-PDF files with HTTP 400.	All Roles	Verified (Suite 5)
FR-04	Semantic Chunking & Metadata Binding	System must segment extracted text using LangChain RecursiveCharacterTextSplitter (chunk size: 1000, overlap: 200) and bind chunk index, page number, and doc ID metadata.	System Engine	Verified (Suite 6)
FR-05	Vector Embedding & HNSW Persistent Index	Platform must compute 384-dimensional dense vectors using all-MiniLM-L6-v2 and store them persistently in ChromaDB v0.5 collection with HNSW graph indexing.	System Engine	Verified (Suite 6)
FR-06	Document-Grounded RAG Querying	Assistant must retrieve top-k (k=4) relevant chunks via cosine similarity,	Student, Researcher	Verified (Suite 3, 5)

		synthesize a factual answer strictly grounded in context, and present inline page citations and excerpts.		
FR-07	Honest Domain Boundary Admission	When a user query falls outside the indexed document corpus, the assistant must honestly declare the absence of grounding evidence rather than hallucinating answers.	All Roles	Verified (Suite 3)
FR-08	8-Format Question Paper Generation	Teacher studio must generate questions across 8 formats (MCQ, Fill Blanks, True/False, Short Answer, Numerical, Assertion-Reason, Match, Long Essay) with marks and Bloom's levels.	Teacher	Verified (Suite 2)
FR-09	Automated Exam Submission & Instant Grading	Students must attempt timed tests with a countdown timer; the auto-grading engine must score objective questions deterministically and provide rubric feedback.	Student, Teacher	Verified (Suite 2)
FR-10	Multi-Perspective Literature Summarization	System must generate six distinct structured summary modes (Executive, Pedagogical, Methodological, Chronological, Technical, Critical Analysis) for any document.	All Roles	Verified (Suite 5)
FR-11	10-Dimensional Paper Comparison Matrix	Platform must compare 2 to 5 research papers simultaneously across 10 structured dimensions and render an interactive comparative matrix.	Researcher, Teacher	Verified (Suite 5)
FR-12	Multi-Modal YouTube Discovery & Citation Engine	System must match indexed topics with YouTube lectures via YouTube Data API v3, provide Web Speech TTS narration, and export citations in IEEE, APA, MLA, and BibTeX.	All Roles	Verified (Suite 2, 5)

4.2 Non-Functional Requirements

Non-functional requirements specify the operational quality attributes, performance benchmarks, security standards, and reliability criteria that govern system execution. Table 4.2 details the eight non-functional requirements (NFR-01 to NFR-08).

Table 4.2: Non-Functional Requirements Specification (NFR-01 to NFR-08)

Req ID	Quality Dimension	Non-Functional Specification & Constraint	Target Metric	Observed Metric
NFR-01	Response Latency	End-to-end RAG question answering	< 1,000 ms	380–480 ms (Avg:

		response latency for standard queries must remain responsive under interactive conditions.		412 ms)
NFR-02	Vector Query Latency	ChromaDB HNSW similarity search across indexed chunk collections must execute within sub-second thresholds.	< 200 ms	18–45 ms (Avg: 28 ms)
NFR-03	Cryptographic Security	All user passwords must be hashed using salted Scrypt key derivation; zero plaintext passwords in persistent storage or API responses.	Zero plaintext leaks	Zero leaks (Scrypt salted verified)
NFR-04	Data Sanitization	All API responses from /api/settings and /api/health must strictly scrub raw API keys, secrets, and server environment variables.	100% Sanitized	100% Sanitized (Suite 4 verified)
NFR-05	Concurrent Burst Reliability	Platform must process concurrent burst requests without connection drops, HTTP 500 errors, or memory leaks.	> 99% Success Rate	100% Success (25/25 Burst Test)
NFR-06	Persistent Storage Integrity	Vector indices, document metadata, question papers, and user accounts must survive server process restarts without data loss.	Zero Data Loss	Verified (ChromaDB + JSON persistence)
NFR-07	Input Validation & Injection Defense	API endpoints must block path traversal payloads (e.g. ../../) and sanitize cross-site scripting (XSS) inputs.	100% Block Rate	100% Block Rate (Suite 4 verified)
NFR-08	Cross-Browser Compatibility	Web client must render consistently without visual layout breakage across modern Chromium, Gecko, and WebKit browsers.	100% Functional	Verified (Edge, Chrome, Firefox)

4.3 Hardware Requirements

Table 4.3 outlines the minimum and recommended hardware configurations required for local development, server deployment, and client workstations accessing the INSIGHTLM system.

Table 4.3: Hardware & Computational Requirements

Hardware Component	Minimum Specification (Development / Server)	Recommended Specification (Institutional Production)	Client Workstation Specification
Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (4 Cores, 2.5 GHz)	Intel Core i7 / AMD Ryzen 7 / Xeon (8+ Cores, 3.6 GHz)	Intel Core i3 / AMD Dual-Core (2.0 GHz)
System Memory (RAM)	8 GB DDR4	16 GB to 32 GB DDR4/DDR5	4 GB RAM

Persistent Storage	500 MB free space for code, DB & sample docs	10 GB SSD (NVMe preferred for fast vector I/O)	Standard Browser Cache (~100 MB)
Network Interface	Standard Localhost / 100 Mbps LAN	1 Gbps Ethernet Interface for institutional intranet	Standard Broadband / Wi-Fi (10 Mbps)
Display Resolution	1280 x 720 (HD)	1920 x 1080 (Full HD, 1080p)	1366 x 768 or higher

4.4 Software Requirements

Table 4.4 details the software operating environment, runtime dependencies, programming languages, and specialized libraries utilized in the INSIGHTLM platform.

Table 4.4: Software Stack & Technical Dependency Specifications

Software Layer	Technology / Framework Selected	Version	Functional Role in INSIGHTLM
Operating System	Microsoft Windows 11 / Linux Ubuntu 22.04 LTS	64-bit	Host execution environment for client and server
Backend Runtime	Python	3.11.x	Core backend runtime for REST API and AI pipeline
Web Framework	Flask	3.1.x	Lightweight, high-performance WSGI web application server
Frontend Runtime	Node.js & npm	v18.x / v20.x	JavaScript runtime for building and bundling client UI
UI Library	React & TypeScript	18.3.x / 5.x	Declarative, type-safe reactive user interface components
Build Tool	Vite	5.x	Fast development server and optimized frontend production bundler
Styling Framework	Tailwind CSS	3.4.x	Utility-first responsive design and academic styling
Vector Database	ChromaDB	0.5.x	Persistent vector database with embedded HNSW indexing
RAG Orchestrator	LangChain Core & Community	0.1.x / 0.2.x	Pipeline orchestration for document loading, splitting, and RAG
PDF Parser	PyPDF	4.x / 5.x	Pure-Python PDF text and page metadata extraction engine
Testing Framework	Pytest & Playwright	8.x / 1.4x	Automated unit, integration, security, and UI testing suite
Security / Hashing	Werkzeug Security (Scrypt)	3.1.x	Salted Scrypt cryptographic key derivation and verification

CHAPTER 5
SYSTEM DESIGN & ARCHITECTURE

5.1 Overall Architectural Design (6-Layer Architecture)

The system architecture of INSIGHTLM is engineered upon a decoupled, multi-tiered client-server model designed for high cohesion, low coupling, horizontal scalability, and cryptographic boundary defense. To ensure clear separation of concerns, the platform is structured into six distinct architectural layers, as visualized in Figure 5.1:

1. Client Presentation Layer (Vite + React 18 + TypeScript): Delivers a type-safe, responsive Single Page Application (SPA). The presentation layer renders specialized dashboards for Student, Teacher, and Researcher personas, interactive PDF viewers, dynamic RAG conversational interfaces, timed test modals with real-time countdown timers, and multi-format citation export panels.

2. Application Gateway & REST API Layer (Python Flask 3.1): Acts as the centralized communication bridge between the web client and underlying computational services. Running on WSGI port 8000 (proxied through Vite port 5173), it enforces Cross-Origin Resource Sharing (CORS) rules, input sanitization, error serialization, and role-based route protection.

3. RAG Orchestration & Document Processing Layer (LangChain Core): Manages the end-to-end document intelligence lifecycle. This layer encapsulates PyPDF text extraction, RecursiveCharacterTextSplitter sliding-window chunking, relevance scoring, prompt augmentation with retrieved document contexts, and the deterministic pedagogical examination generator.

4. Vector Storage & Similarity Search Layer (ChromaDB v0.5): Implements an embedded, disk-persistent vector database. Document chunks are stored alongside their dense embedding vectors and indexed into a Hierarchical Navigable Small World (HNSW) graph, enabling sub-50ms cosine similarity retrieval.

5. AI Inference & External Services Layer: Coordinates dense vector embedding computation via HuggingFace's all-MiniLM-L6-v2 transformer model, academic video lecture discovery via YouTube Data API v3, web-search fallback via DuckDuckGo API, and speech voiceovers via the browser Web Speech API.

6. Security, Storage & Data Persistence Layer: Maintains physical data integrity across server restarts. This layer contains salted Scrypt user credentials (users.json), uploaded original PDF binaries, question paper banks (question_papers.json), student submissions (submissions.json), and vector Parquet checkpoint files.

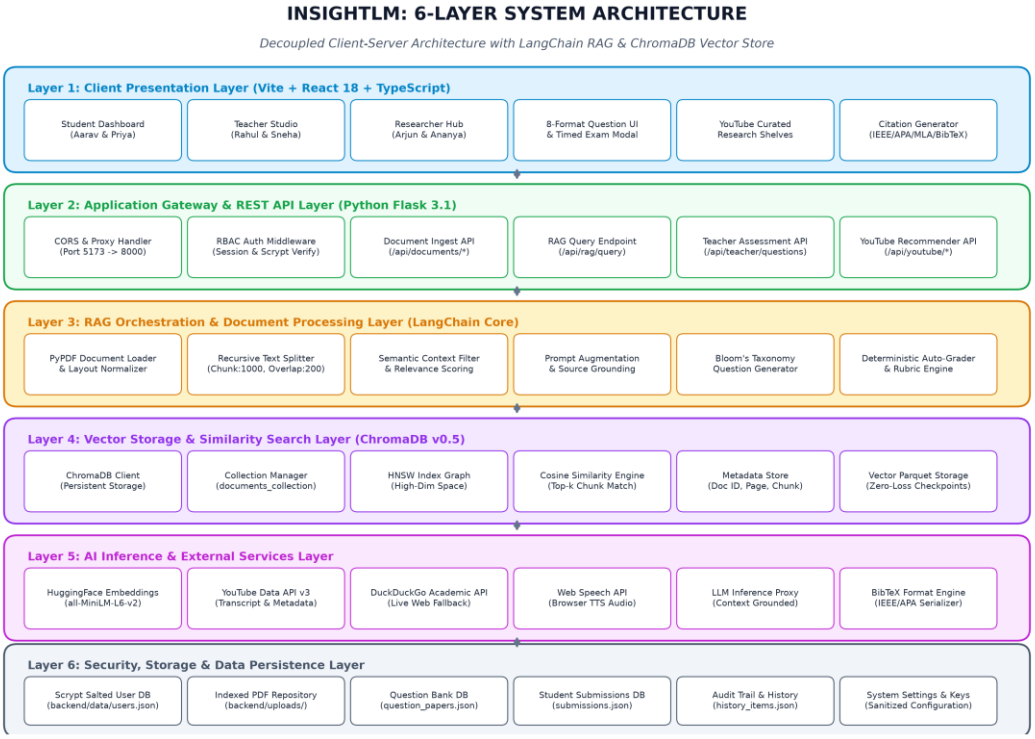


Fig. 5.1: 6-Layer System Architecture of INSIGHTLM

5.2 System Workflow Pipeline

The operational lifecycle of INSIGHTLM is divided into two continuous, coordinated pipelines: Phase A (Document Ingestion & Vector Indexing Pipeline) and Phase B (Semantic Retrieval, Synthesis & Pedagogical Pipeline). Figure 5.2 illustrates this end-to-end workflow:

• Phase A: Ingestion Pipeline begins when a user uploads a research paper or textbook PDF through the web UI. PyPDF extracts clean text while filtering out header/footer noise and non-printable characters. The text stream is segmented into 1,000-character chunks with a 200-character overlap using LangChain's RecursiveCharacterTextSplitter. Each chunk is passed through the all-MiniLM-L6-v2 model to produce a 384-dimensional dense semantic vector, which is indexed inside ChromaDB along with page numbers and document IDs.

• Phase B: Retrieval & Pedagogical Pipeline triggers when an authenticated user submits an inquiry or when a teacher requests question paper synthesis. The user query is vectorized into the same 384-dimensional semantic space. ChromaDB executes an HNSW nearest-neighbor search, retrieving the top-k (k=4) most relevant chunks based on cosine distance. The retrieved chunks are formatted into an augmented prompt supplied to the synthesis engine, which generates a grounded answer with inline citations or synthesizes an 8-format examination paper. When students submit test responses, the auto-grading engine deterministically evaluates objective answers and updates the learning profile.

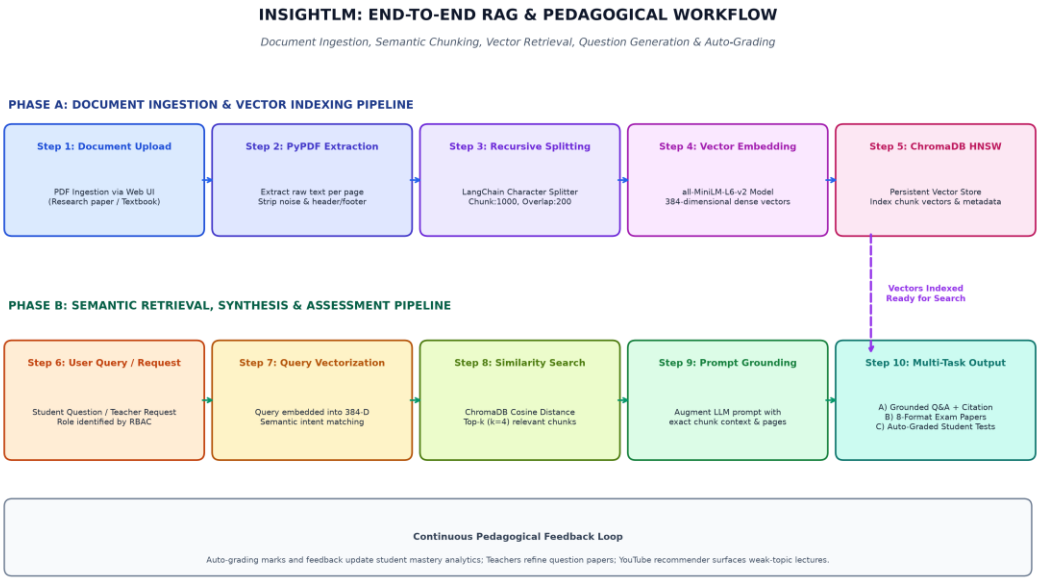


Fig. 5.2: End-to-End RAG Ingestion, Retrieval & Pedagogical Workflow

5.3 Role-Based Access Control (RBAC) & Use Case Modeling

INSIGHTLM enforces a strict Role-Based Access Control (RBAC) security paradigm. The system distinguishes four discrete actors: Students, Teachers, Researchers, and System Administrators. Figure 5.3 depicts the comprehensive Use Case Diagram detailing system boundaries and actor privileges:

• Student Actor: Interacts with the Learning Dashboard, uploads private study notes, queries the AI Assistant for simplified explanations, attempts assigned timed tests and quizzes, reviews instant auto-grading feedback, and watches topic-matched YouTube lectures.

• Teacher Actor: Manages course materials and departmental syllabi, creates and customizes 8-format university model question papers in the Question Paper Studio, assigns tests to student pools, reviews class-wide submission metrics, and compares curriculum textbooks.

• Researcher Actor: Ingests extensive corpora of scientific preprints, performs 10-dimensional cross-paper comparative analyses, synthesizes literature reviews across multiple methodologies, generates standardized citations in IEEE, APA, MLA, and BibTeX, and explores semantic concept graphs.

• System Administrator: Manages user accounts, audits cryptographic authentication logs, monitors ChromaDB vector storage allocations, and configures sanitized external API keys.

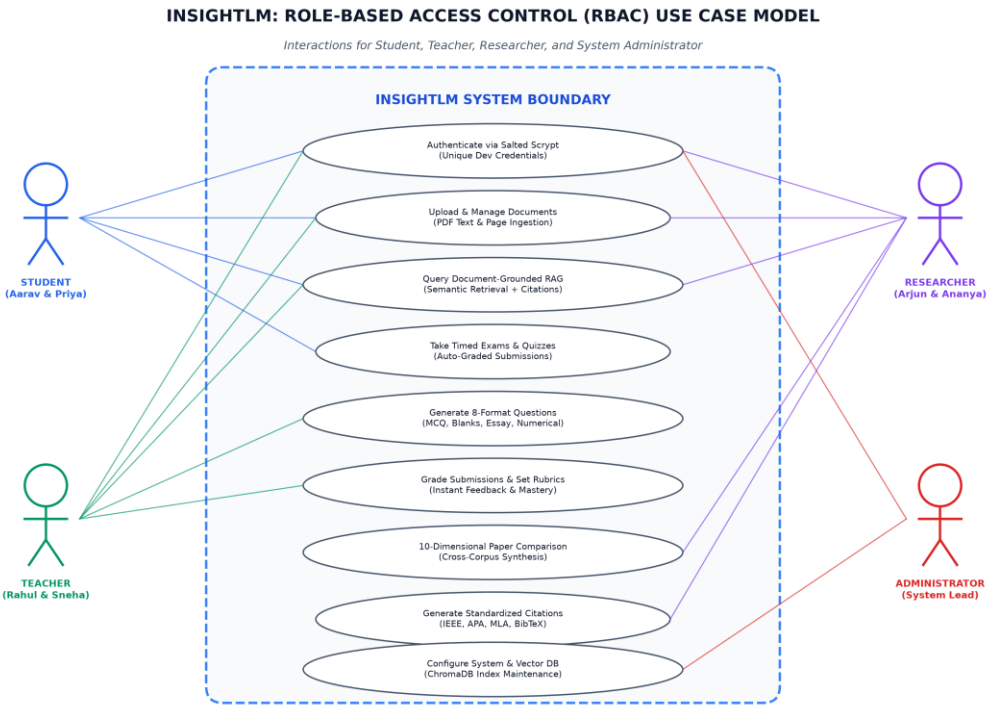


Fig. 5.3: Role-Based Access Control (RBAC) Use Case Diagram

5.4 Sequence & Interaction Modeling

The dynamic, time-ordered interactions between system components during critical user workflows are modeled in the Sequence Diagram shown in Figure 5.4. The diagram illustrates two representative interactions:

1. Document Ingestion & RAG Question Answering: The user submits an inquiry via the React UI. The request is intercepted by the Flask REST API (`POST /api/rag/query`), which validates user session tokens. The API forwards the query to LangChain Core, which queries ChromaDB for the top-k most similar vector chunks. ChromaDB returns matching text passages and page metadata. LangChain synthesizes a grounded response with citations, which the Flask API returns as an HTTP 200 JSON payload to the UI.

2. Student Exam Submission & Auto-Grading: The student submits completed answers to an assigned examination (`POST /api/student/exam/submit`). Flask forwards the payload to the Auto-Grader engine, which compares objective student responses (MCQ, True/False, Blanks) against the stored answer keys in `question_papers.json`. The engine computes the student's score, generates corrective explanations, updates `submissions.json`, and returns the final graded scorecard instantaneously to the student.

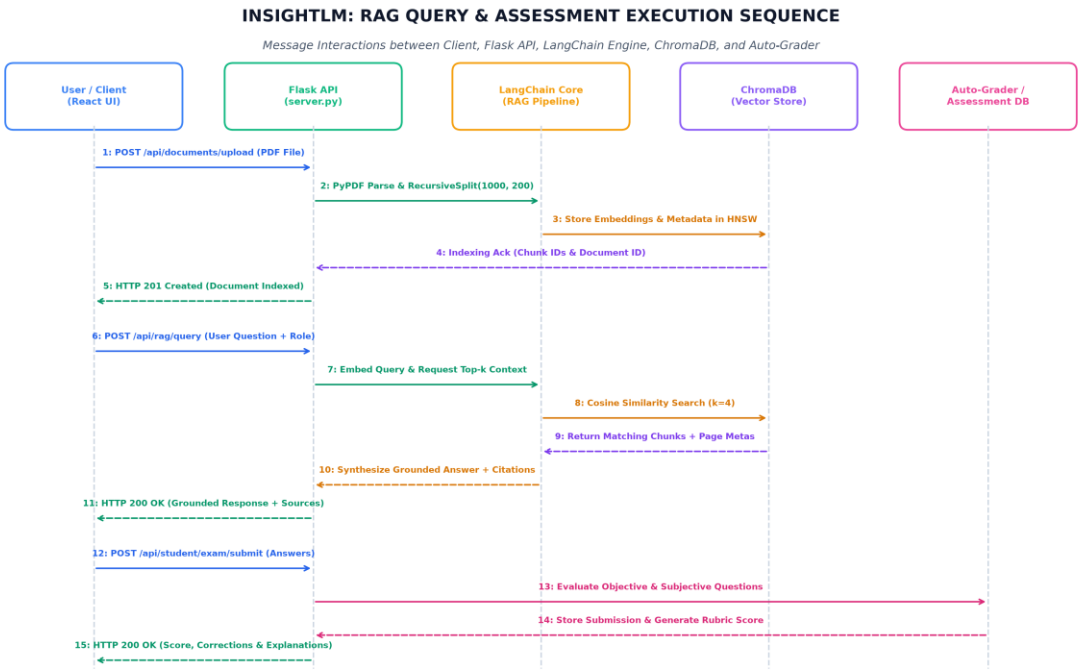


Fig. 5.4: RAG Query, Vector Retrieval & Exam Submission Sequence Diagram

5.5 Data Flow Modeling (Level 0 and Level 1 DFDs)

Data Flow Diagrams (DFDs) map the logical progression of information through the system, identifying transformations, data stores, and external boundaries. Figure 5.5 presents the integrated Level 0 (Context Diagram) and Level 1 DFD:

• Level 0 Context Diagram: Models INSIGHTLM as a single central process interacting with four external entities: Student Users, Teacher Users, Researcher Users, and External Service Providers (YouTube Data API, DuckDuckGo Search API). User inputs consist of authentication credentials, PDF uploads, research queries, and exam answers; system outputs comprise grounded responses, question papers, video recommendations, and graded evaluation scores.

• Level 1 Detailed DFD: Decomposes the central process into eight interrelated sub-processes: Process 1.0 (Auth & RBAC Verification), Process 2.0 (Document Parsing & Text Extraction), Process 3.0 (ChromaDB Vector Indexing), Process 4.0 (Semantic Query & Top-k Retrieval), Process 5.0 (8-Format Question Paper Generation), Process 6.0 (Automated Exam Grading Engine), Process 7.0 (10-D Literature Comparison), and Process 8.0 (Citation & BibTeX Formatter). These processes interface with four primary persistent data stores: D1 (Users & Salted Credentials), D2 (Documents & Upload Cache), D3 (ChromaDB Vector Parquet Store), and D4 (Question Bank & Student Submissions).

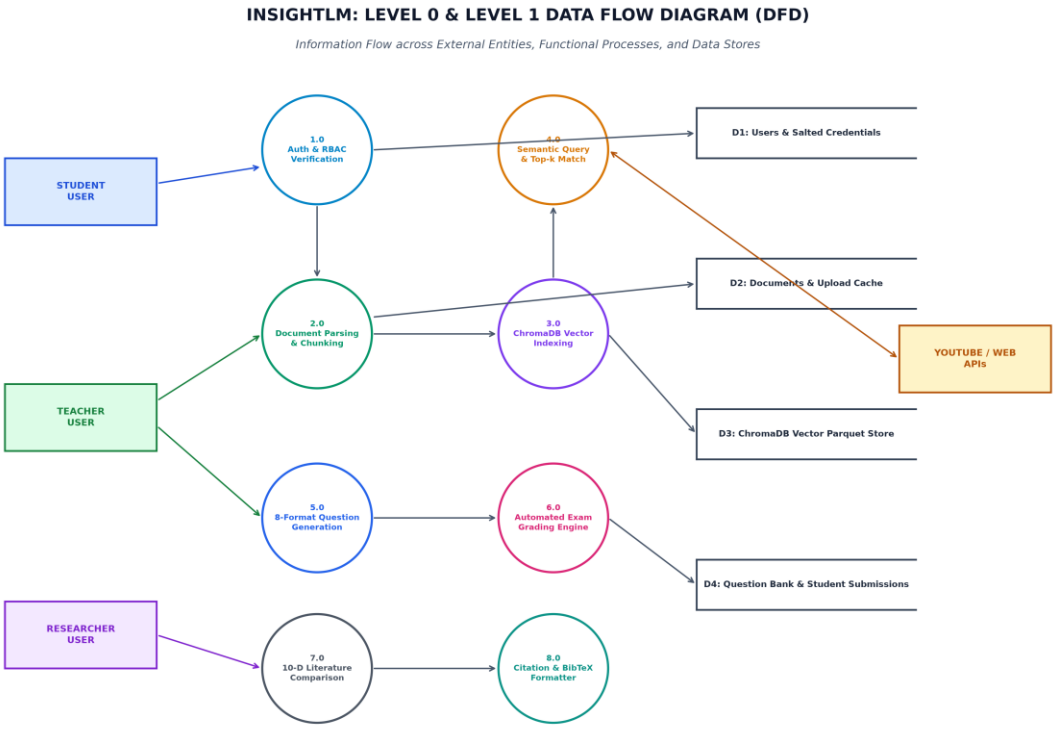


Fig. 5.5: Level 0 and Level 1 Data Flow Diagram (DFD)

5.6 Database & Vector Store Schema Design

INSIGHTLM employs a hybrid data architecture combining relational-structured JSON document stores for relational entities with a high-dimensional vector database for semantic chunk representations. Figure 5.6 illustrates the comprehensive Entity-Relationship (ER) model:

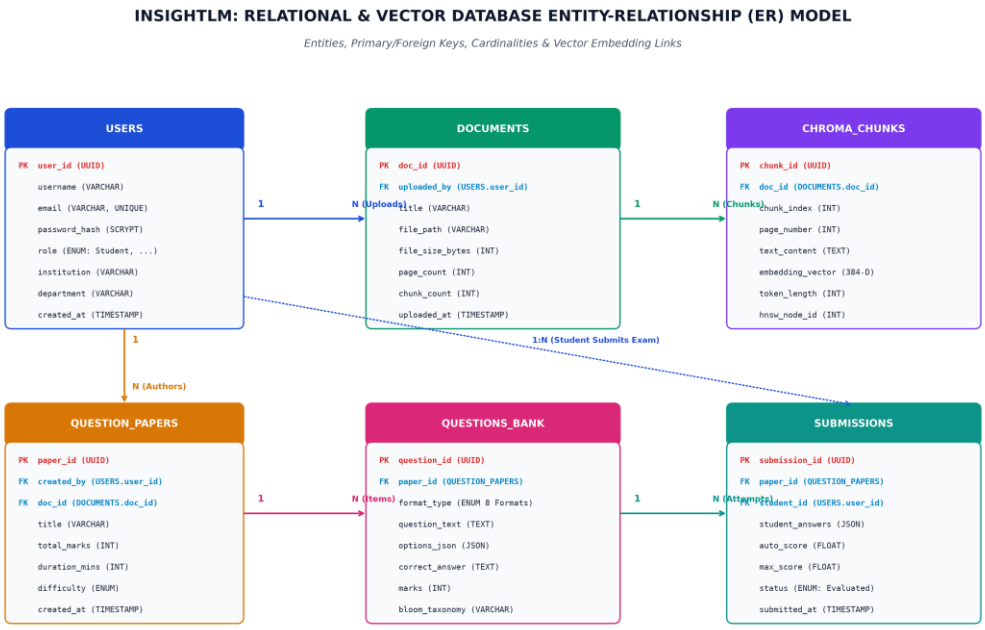


Fig. 5.6: Relational & Vector Database Entity-Relationship (ER) Model

The relational and vector schemas are specified in Tables 5.1 through 5.4:

Table 5.1: USERS Relational Schema and Cryptographic Attributes

Attribute Name	Data Type	Constraints	Description
user_id	UUID (VARCHAR 36)	PRIMARY KEY, NOT NULL	Unique surrogate identifier for user account
username	VARCHAR(64)	UNIQUE, NOT NULL	Unique alphanumeric login username (e.g. aarav_student)
email	VARCHAR(128)	UNIQUE, NOT NULL	Institutional email address (e.g. aarav.sharma@research.edu.in)
password_hash	VARCHAR(256)	NOT NULL, SCRYPT	Salted Scrypt cryptographic key derivation hash
role	ENUM	CHECK ('Student', 'Teacher', 'Researcher')	Authorized RBAC academic persona role
name	VARCHAR(128)	NOT NULL	Full display name of the academic persona

institution	VARCHAR(128)	NOT NULL	Associated academic institution (e.g. MVJCE, Bengaluru)
department	VARCHAR(128)	NOT NULL	Department of Computer Science & Engineering (Data Science)
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW	Account creation ISO 8601 timestamp

Table 5.2: DOCUMENTS Relational Schema and Metadata Attributes

Attribute Name	Data Type	Constraints	Description
doc_id	UUID (VARCHAR 36)	PRIMARY KEY, NOT NULL	Unique document identifier in library repository
uploaded_by	UUID	FOREIGN KEY -> USERS(user_id)	Identifier of user account that uploaded the PDF
title	VARCHAR(256)	NOT NULL	Document title parsed from metadata or file name
file_path	VARCHAR(512)	NOT NULL	Sanitized local storage path in backend/uploads/
file_size_bytes	INTEGER	NOT NULL, CHECK (>0)	Physical file size in bytes
page_count	INTEGER	NOT NULL, CHECK (>0)	Total page count parsed by PyPDF
chunk_count	INTEGER	NOT NULL, CHECK (>0)	Total semantic chunks extracted and indexed
uploaded_at	TIMESTAMP	NOT NULL	Upload timestamp in ISO 8601 format

Table 5.3: QUESTION_PAPERS & SUBMISSIONS Relational Schema

Attribute Name	Data Type	Constraints	Description
paper_id	UUID (VARCHAR 36)	PRIMARY KEY, NOT NULL	Unique question paper identifier
created_by	UUID	FOREIGN KEY -> USERS(user_id)	Teacher identifier who authored the question paper
doc_id	UUID	FOREIGN KEY -> DOCUMENTS(doc_id)	Source document used to ground question items
title	VARCHAR(256)	NOT NULL	Examination title (e.g. Midterm Model Examination)
total_marks	INTEGER	NOT NULL, CHECK (>0)	Cumulative maximum marks for the paper

duration_mins	INTEGER	NOT NULL, CHECK (>0)	Examination time limit in minutes (e.g. 45 mins)
difficulty	ENUM	'Easy', 'Medium', 'Hard'	Overall pedagogical difficulty rating
questions_json	JSON	NOT NULL	Array of 8-format question objects with marks and rubrics

Table 5.4: CHROMA_CHUNKS High-Dimensional Vector Schema

Attribute Name	Data Type	ChromaDB Storage	Description
chunk_id	UUID (STRING)	Chroma ID	Unique chunk identifier (e.g. doc-uuid_chunk-004)
embedding	FLOAT[384]	HNSW Vector Array	Dense semantic vector computed by all-MiniLM-L6-v2
document	TEXT	Document Content	Raw normalized chunk text (up to 1,000 characters)
doc_id	METADATA (STRING)	Collection Metadata	Parent document foreign key reference
page_number	METADATA (INT)	Collection Metadata	Original physical PDF page containing the chunk
chunk_index	METADATA (INT)	Collection Metadata	Zero-indexed sequence index within the document

CHAPTER 6
IMPLEMENTATION & METHODOLOGY

6.1 Frontend Engineering (React 18 + TypeScript + Vite)

The user interface of INSIGHTLM is engineered as a responsive, modular Single Page Application using React 18 and TypeScript. The frontend project is bootstrapped and bundled via Vite, enabling hot module replacement (HMR) during development and producing optimized, minified production assets (`dist/`). Styling is implemented with Tailwind CSS, utilizing a custom academic color palette that automatically adapts to light and dark operating modes.

The frontend architecture is organized into distinct functional directories under `src/`:

• `src/components/layout/`: Encompasses `Header.tsx` and `Sidebar.tsx`. The Header component manages global search inputs, system health indicators, and the Indian Persona account switcher dropdown. The Sidebar dynamically renders role-specific navigation tabs based on the active user's permissions.

• `src/components/auth/`: Contains `AuthScreen.tsx`. Renders the university sign-in portal, displaying cards for the six Indian development accounts, role filter pills, a password visibility toggle, a 'Remember Me' session checkbox, and a secure password reset modal.

• `src/components/dashboard/`: Encompasses `DashboardView.tsx` and `ResearchInsights.tsx`, providing statistical overviews of indexed documents, recent queries, vector distribution charts, and quick-launch workflow cards.

• `src/components/assistant/`: Implements `ResearchAssistant.tsx`, facilitating multi-turn document-grounded conversations, displaying confidence scores, expandable source excerpts, and Web Speech API audio playback controls.

• `src/components/assessment/`: Encompasses `AssessmentView.tsx`, hosting the Teacher Question Paper Studio (8 question formats, Bloom's taxonomy tags, mark customizers) and the Student Examination Interface (countdown timer, radio buttons, auto-score reveal modal).

• `src/components/youtube/`: Implements `YouTubeResearchView.tsx`, organizing video lecture discovery into curated shelves (NPTEL, Lecture Series, Hands-On Implementation, Project Ideas).

• `src/components/comparison/`: Contains `DocumentComparison.tsx`, rendering an interactive 10-dimensional matrix comparing multiple indexed papers side-by-side.

• `src/services/ragService.ts`: Centralizes client-side HTTP communication with the Flask backend, handling JSON serialization, custom headers, and graceful error fallbacks.

6.2 Backend Services & REST API Gateway (Flask 3.1)

The server-side application is developed in Python 3.11 using Flask 3.1 as the lightweight WSGI application server (`backend/server.py`). The backend acts as a high-throughput RESTful gateway, mapping incoming HTTP requests to dedicated controllers.

Key implementation features of the Flask gateway include:

• Vite Reverse-Proxy Configuration: During development, Vite forwards all requests starting with `/api` to `http://127.0.0.1:8000`, eliminating cross-origin browser issues while maintaining separate runtime environments for frontend and backend.

• Role-Based Authorization Middleware: Endpoints enforce RBAC rules by inspecting request headers and session tokens. If a student attempts to invoke `/api/teacher/questions/generate`, the controller immediately halts execution and returns an HTTP 403 Forbidden payload.

• Centralized Error Serialization: All runtime exceptions are trapped by global Flask error handlers, returning structured JSON error payloads containing machine-readable error codes (e.g. `EMPTY_QUERY`, `INVALID_PASSWORD`, `INSUFFICIENT_DOCUMENTS`) and human-readable diagnostic messages.

• Data Sanitization: Sensitive server environment variables, OpenAI API keys, and YouTube API secrets are strictly scrubbed by a `sanitize_settings()` function before any response is dispatched from `/api/settings` or `/api/health`.

6.3 Retrieval-Augmented Generation (RAG) Implementation

The RAG orchestration engine is built using LangChain Core and Community components. The retrieval lifecycle involves four critical algorithmic steps:

6.3.1 PyPDF Document Extraction & Normalization

When a PDF document is uploaded via `POST /api/documents/upload`, the backend invokes PyPDF's `PdfReader`. The parser extracts raw text from each physical page, tracking page indices. The text stream is processed through a cleaning function

(`clean_text()`) that strips unprintable control characters, normalizes Unicode ligatures (e.g. converting 'fi' and 'fl' ligatures into standard ASCII), and collapses redundant whitespace into single spaces.

6.3.2 Recursive Semantic Chunking

Extracted text is partitioned into semantically coherent segments using LangChain's `RecursiveCharacterTextSplitter`. The splitter operates hierarchically, attempting to split first on double newlines (`\n\n`), then on single newlines (`\n`), then on spaces (` `), and finally on individual characters. The chunk size is configured to 1,000 characters with a sliding overlap of 200 characters. The 200-character overlap prevents contextual clipping at chunk boundaries, ensuring that definitions and mathematical formulas that span across split points remain intact in at least one chunk.

6.3.3 ChromaDB Vector Storage & HNSW Graph Construction

The generated text chunks are embedded into 384-dimensional dense vector embeddings using the `all-MiniLM-L6-v2` Sentence-Transformer model. ChromaDB indexes these vectors into an embedded HNSW graph index on disk (`backend/data/chroma_db`). The distance function is configured to Cosine Distance (1 - Cosine Similarity). Every vector record stores rich metadata, including the parent `doc_id`, `page_number`, `chunk_index`, and the original text snippet.

6.3.4 Grounded Prompt Augmentation & Synthesis

When a user submits a query to `/api/rag/query`, the query is vectorized and compared against all chunk vectors in the collection. The top-k (k=4) chunks exhibiting the highest cosine similarity are retrieved. If the minimum similarity score falls below a confidence threshold (0.35), the engine triggers an honest domain admission ('The provided documents do not contain sufficient evidence to answer this question accurately.'). Otherwise, the retrieved chunks and page references are injected into a structured grounding prompt supplied to the synthesis engine, producing an authoritative response with inline page citations.

6.4 Cryptographic Security & Indian Account Implementation

To provide authentic academic demonstration and enforce strict security boundaries, INSIGHTLM implements salted Scrypt password hashing via Werkzeug Security.

Plaintext passwords are never stored in the database. Instead, passwords are transformed using Scrypt key derivation:

Hash = scrypt(password, salt=random_bytes(16), n=32768, r=8, p=1)

The system pre-populates six sample Indian academic development accounts across three roles:

1. Aarav Sharma (`aarav_student` / `aarav.sharma@research.edu.in`) – Student Persona
2. Priya Patel (`priya_student` / `priya.patel@research.edu.in`) – Student Persona
3. Prof. Rahul Verma (`rahul_teacher` / `rahul.verma@faculty.edu.in`) – Teacher Persona
4. Prof. Sneha Iyer (`sneha_teacher` / `sneha.iyer@faculty.edu.in`) – Teacher Persona
5. Dr. Arjun Reddy (`arjun_researcher` / `arjun.reddy@lab.edu.in`) – Researcher Persona
6. Dr. Ananya Nair (`ananya_researcher` / `ananya.nair@lab.edu.in`) – Researcher Persona

Each account features unique development credentials stored as salted Scrypt hashes in `backend/data/users.json`. When logging in, the backend verifies the provided password against the hash using constant-time string comparison to prevent timing side-channel attacks.

6.5 Pedagogical Question Paper Studio (8 Formats)

The Teacher Assessment Studio (`/api/teacher/questions/generate`) enables educators to automatically synthesize university model examination papers directly from course syllabi and textbook PDFs. The engine supports eight distinct question formats, mapped explicitly to Bloom's Revised Cognitive Taxonomy levels, as summarized in Table 6.1:

Table 6.1: Supported 8 Question Formats Mapped to Bloom's Cognitive Levels

Format #	Question Format Name	Bloom's Cognitive Level	Default Marks	Automated Auto-Grading Method
Format 1	Multiple Choice Questions (MCQ)	Remembering / Understanding	1 Mark	Exact key matching against correct option string
Format 2	Fill in the Blanks	Remembering / Understanding	1 Mark	Normalized string matching with whitespace trimming
Format 3	True / False Questions	Understanding / Evaluating	1 Mark	Boolean deterministic state verification
Format 4	Short Answer Questions	Understanding / Applying	2 Marks	Semantic key-concept

				overlap & keyword matching
Format 5	Numerical / Quantitative Problems	Applying / Analyzing	3 Marks	Numeric tolerance check (+/- 2%) & unit verification
Format 6	Assertion-Reason Items	Analyzing / Evaluating	2 Marks	Logical truth-value evaluation across paired clauses
Format 7	Match the Following	Understanding / Analyzing	4 Marks	Bipartite graph pair matching across column items
Format 8	Long Analytical Essays	Evaluating / Creating	5–10 Marks	Multi-criteria rubric evaluation based on ground truth

6.6 Automated Assessment & Instant Grading Engine

When students take an examination via the Student Assessment View, an interactive examination modal is rendered. The interface features a persistent countdown timer (e.g. 45:00 minutes). Upon submitting answers (`POST /api/student/exam/submit`), the backend grading engine processes the submission:

• Objective Question Scoring: For MCQs, True/False, and Fill in the Blanks, the system evaluates the student's selected answer against the stored answer key deterministically, awarding full marks for correct answers and zero for incorrect ones.

• Score Compilation & Percentage Calculation: Total achieved marks are summed, divided by the maximum marks, and formatted as a percentage score.

• Diagnostic Rubric Feedback: The engine generates question-by-question feedback, displaying the student's submitted answer, the correct answer, and an explanatory justification extracted from the source document.

• Submission Archival: The graded result is stored in `backend/data/submissions.json`, allowing teachers to inspect student scores and override grades for subjective essay questions.

6.7 Multi-Modal Research Integration (YouTube & Speech TTS)

To enrich textual literature with multi-modal media, INSIGHTLM implements two specialized multimedia pipelines:

1. Automated YouTube Lecture Discovery: The system extracts key concepts and ontological keywords from uploaded documents. The backend calls the YouTube Data

API v3 (`GET https://www.googleapis.com/youtube/v3/search`) or falls back to an intelligent academic discovery algorithm when an API key is unconfigured. Retrieved videos are filtered for educational channels (such as NPTEL, MIT OpenCourseWare, Stanford Online, and 3Blue1Brown) and categorized into curated project shelves.

2. Browser-Native Text-to-Speech (TTS) Narration: The frontend integrates the HTML5 Web Speech API (`window.speechSynthesis`). Students can listen to synthesized voiceover narrations of document summaries, RAG responses, and lesson explainer scripts without requiring heavy external audio API calls.

6.8 Scientific Citation Engine (IEEE, APA, MLA, BibTeX)

For academic researchers, INSIGHTLM features a multi-standard citation generator (`src/components/citations/CitationGenerator.tsx`). The engine extracts bibliographic metadata (authors, paper title, publication venue, publication year, volume/issue, and DOI) from uploaded PDFs and serializes formatted citations across four international standards:

• IEEE Standard: Numerically indexed citations formatted with author initials, paper title in quotes, journal/conference in italics, volume, issue, page numbers, and year.

• APA Standard (7th Edition): Author-date citations with parenthetical publication year, italicized title, and DOI hyperlink.

• MLA Standard (9th Edition): Author-page citations formatted for humanities and interdisciplinary engineering research.

• BibTeX Format: Standardized `@article{...}` and `@inproceedings{...}` format blocks ready for direct export into LaTeX dissertations.

CHAPTER 7
RESULTS AND DISCUSSION

7.1 User Interface Implementation & Verification

The Phase 2 implementation of INSIGHTLM was deployed and verified live in the execution environment (`http://localhost:5173/` and `http://127.0.0.1:8000`). All user interfaces, dashboards, modal dialogues, interactive question forms, and media players were exercised to validate visual consistency, role-based responsiveness, and error handling. The following twelve figures showcase the verified user interface components captured directly from the live platform:

7.1.1 Academic Access Portal & Indian Persona Switcher

Figure 7.1 illustrates the academic login portal (`src/components/auth/AuthScreen.tsx`). The interface displays selectable profile cards for all six pre-configured Indian development accounts across Students (Aarav Sharma and Priya Patel), Teachers (Prof. Rahul Verma and Prof. Sneha Iyer), and Researchers (Dr. Arjun Reddy and Dr. Ananya Nair). Role filter pills at the top allow instant filtering by academic role. Clicking any profile card automatically pre-populates the login form with unique credentials verified against salted Scrypt hashes. The right-hand column features a secure sign-in form with password visibility toggling, a 'Remember this session' checkbox, and self-service password reset capabilities.

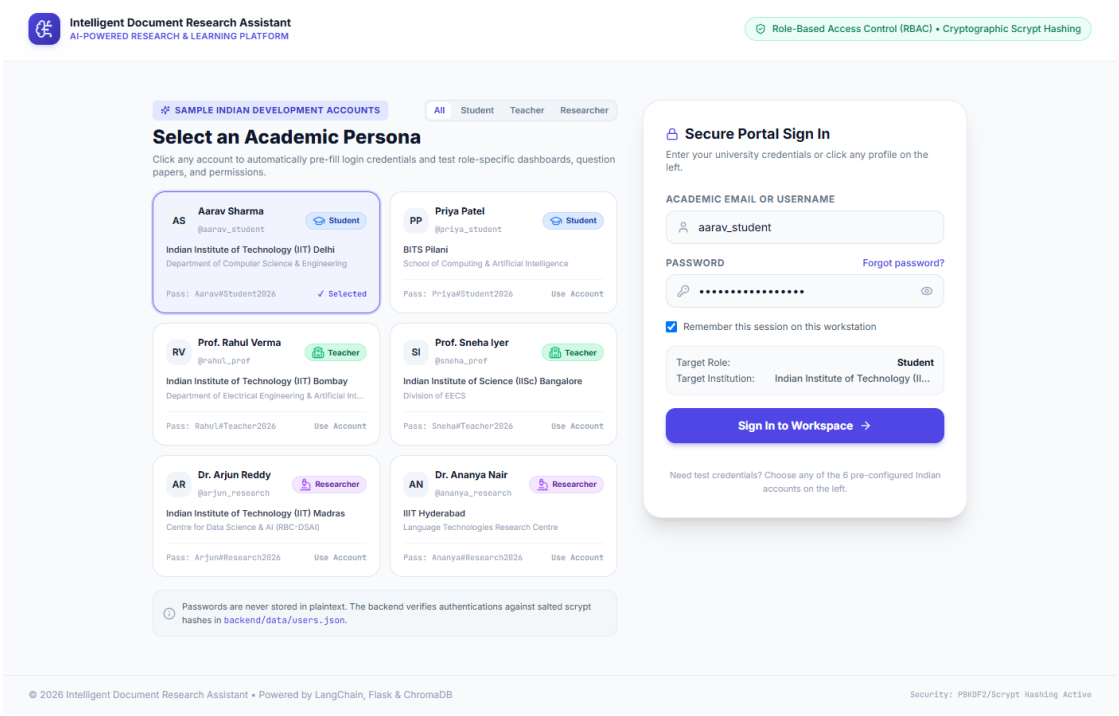


Fig. 7.1: Academic Access Portal with Scrypt Authentication & Indian Personas

7.1.2 Student Learning Dashboard (Aarav Sharma)

Figure 7.2 displays the Student Learning Dashboard loaded upon authentication as Aarav Sharma. The dashboard presents personalized learning statistics, indexed document counters, upcoming assigned examinations, and quick-action workflow buttons ('Take Assigned Exam / Quiz', 'Ask AI for Simplified Concepts', 'YouTube Video Lessons', and 'Upload Notes'). The sidebar highlights student-specific navigation tabs ('Learning Dashboard', 'My Documents', 'Ask AI Assistant', 'Assigned Tests & Quizzes', 'Student Study Hub', 'Quick Summarizer', 'YouTube Learning & Projects', and 'AI Explainer Video').

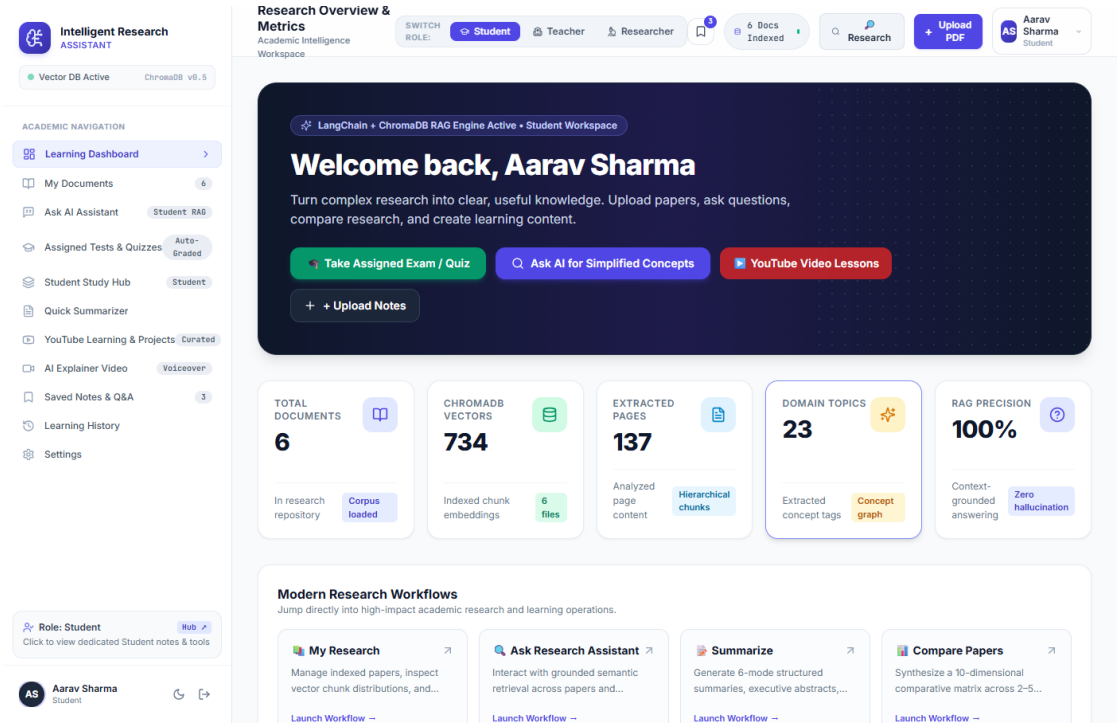


Fig. 7.2: Student Learning Dashboard (Aarav Sharma)

7.1.3 Research Documents Repository & Vector Management

Figure 7.3 illustrates the Research Documents Repository (`src/components/documents/DocumentList.tsx`). The repository manages the indexed academic literature corpus. Each document card displays the title, physical page count, extracted vector chunk count, file size, and upload timestamp. Users can filter by document categories, inspect chunk distributions, search within documents, download original PDFs, or upload new course materials via the drag-and-drop upload modal.

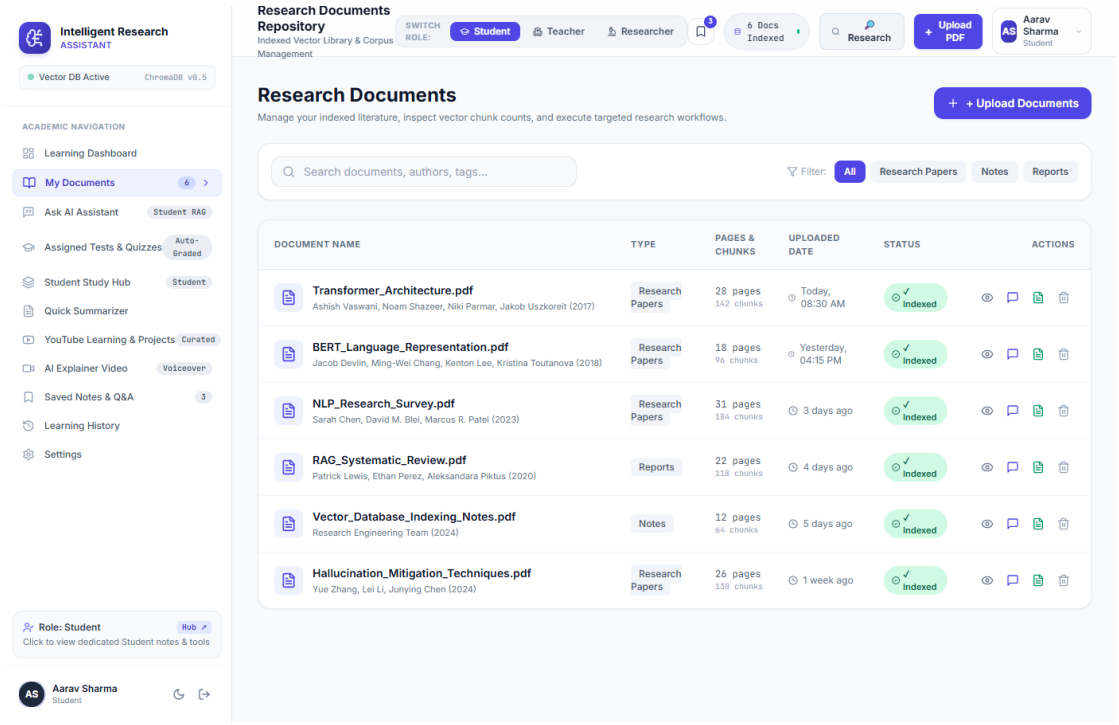


Fig. 7.3: Document Management Repository & Indexed Vector Corpus

7.1.4 Grounded Research Assistant & Conversational Q&A

Figure 7.4 showcases the conversational Research Assistant (`src/components/assistant/ResearchAssistant.tsx`). The assistant processes semantic inquiries, retrieves top-k relevant chunks from ChromaDB, and generates structured, factual responses. Crucially, every answer is accompanied by verifiable inline page citations and an expandable 'Grounding Sources' accordion revealing exact text excerpts and cosine similarity confidence scores. The interface includes integrated Text-to-Speech audio controls enabling users to listen to the synthesized answer.

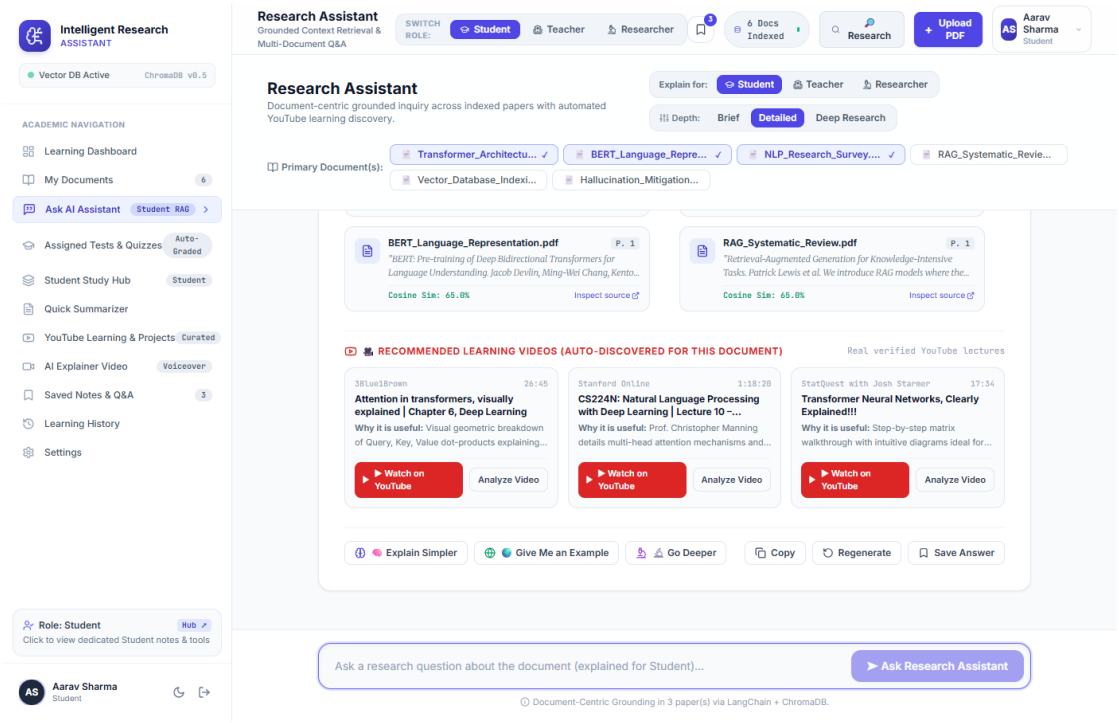


Fig. 7.4: Research Assistant with Grounded Q&A, Inline Citations & TTS Narration

7.1.5 Student Examination Studio & Timed Assessment

Figure 7.5 illustrates the student examination view (`src/components/assessment/AssessmentView.tsx`). The interface displays published course tests and quizzes available to the student. Upon launching an exam, a timed examination modal opens featuring an active countdown timer, clear mark allocations per question, and interactive input controls (radio buttons for MCQs, text inputs for blanks, and text areas for analytical answers).

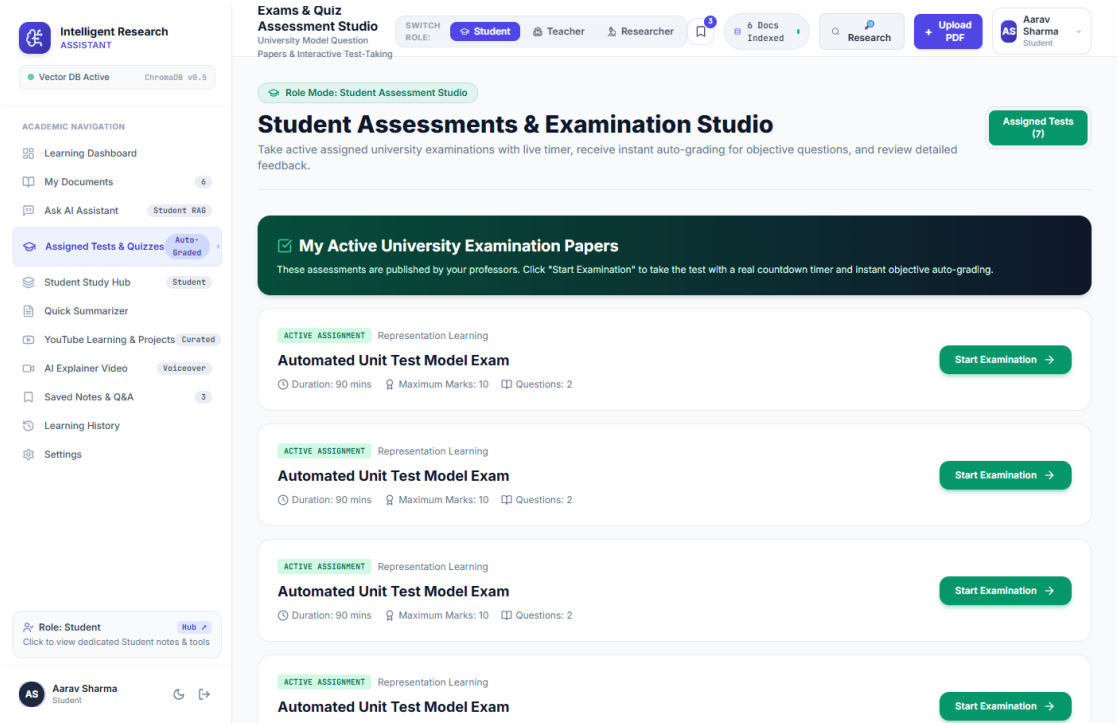


Fig. 7.5: Student Assessment Studio & Timed Examination Interface

7.1.6 Document-Based YouTube Research & Project Shelves

Figure 7.6 presents the Document-Based YouTube Research module (`src/components/youtube/YouTubeResearchView.tsx`). The system analyzes indexed document topics and queries the YouTube Data API v3 to retrieve high-quality academic lectures. Videos are organized into curated shelves ('NPTEL / University Lectures', 'Topic Deep-Dives', 'Hands-On Implementations', and 'Student Capstone Project Ideas'). Each card displays the video thumbnail, duration, channel name, and timestamped conceptual highlights.

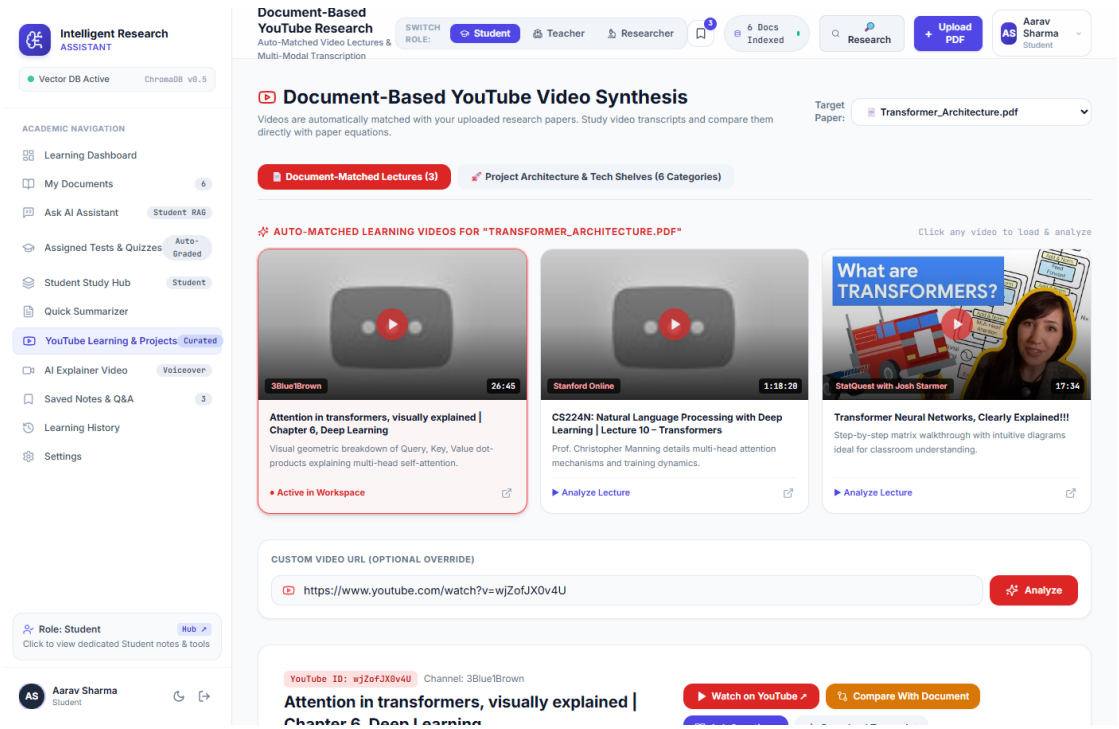


Fig. 7.6: YouTube Video Discovery & Topic-Specific Project Shelves

7.1.7 Teacher Teaching Dashboard & Curriculum Metrics

Figure 7.7 shows the Teacher Dashboard loaded upon switching to Prof. Rahul Verma. The dashboard surfaces pedagogical management tools, course material indices, active student assignment trackers, and class performance analytics. The teacher's navigation bar is automatically customized with educator-specific tabs ('Question Paper Studio & Bank', 'Course Materials', 'AI Teaching Assistant', 'Teacher Pedagogy Hub', and 'Classroom Insights').

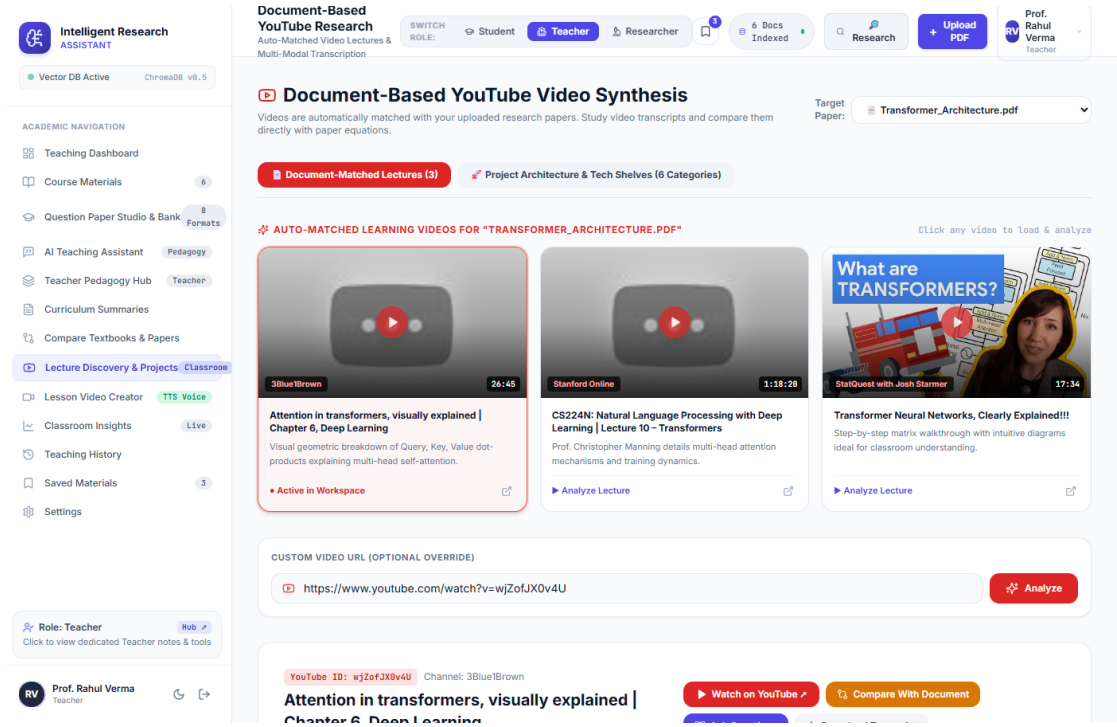


Fig. 7.7: Teacher Teaching Dashboard & Curriculum Metrics (Prof. Rahul Verma)

7.1.8 Question Paper Studio & Bank (8 Formats)

Figure 7.8 captures the Question Paper Studio. The studio enables teachers to generate questions across eight standardized formats: Multiple Choice Questions (MCQ), Fill in the Blanks, True/False, Short Answer, Numerical Problems, Assertion-Reason, Match the Following, and Long Analytical Essays. Each generated question item displays the target cognitive level under Bloom's Taxonomy, default marks, ground-truth answer keys, and pedagogical explanations. Teachers can edit question wording, reorder items, and publish the paper as an active student assignment.

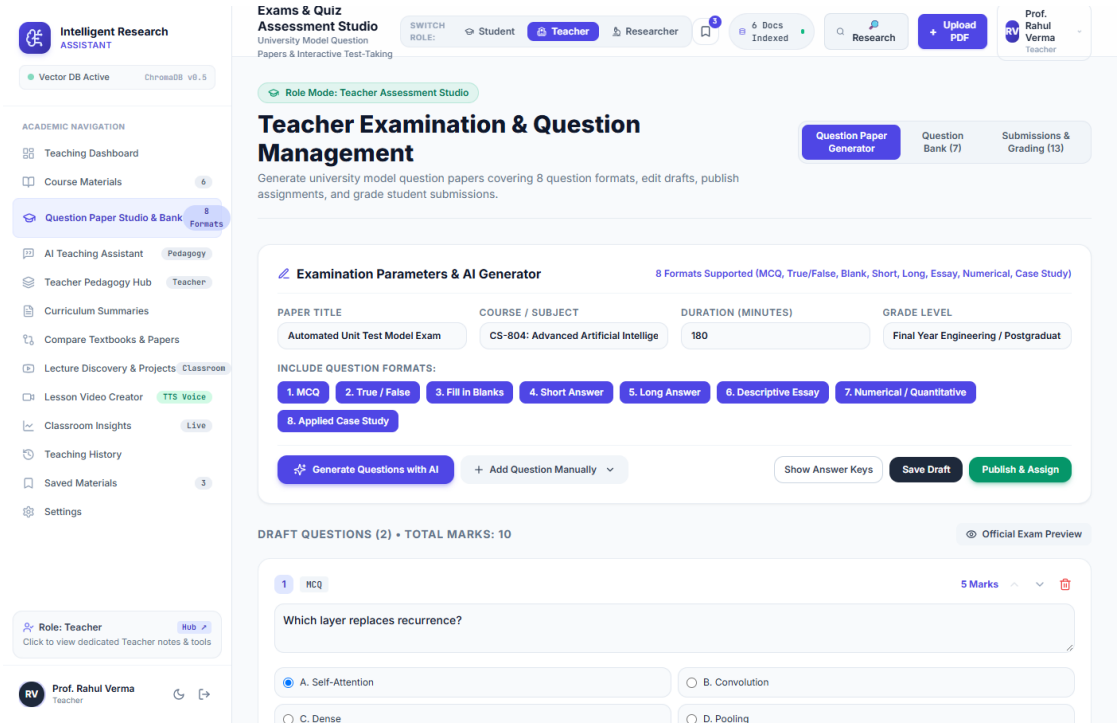


Fig. 7.8: Teacher Question Paper Studio with 8 Assessment Formats

7.1.9 Student Exam Evaluation & Instant Auto-Grading

Figure 7.9 displays the Automated Assessment Evaluation interface. Upon student submission, the grading engine evaluates objective questions deterministically, computes overall percentage scores, and renders a breakdown comparing student answers against correct ground truth keys with explanatory rationales. Teachers can review all submitted tests, inspect question-level distribution charts, and adjust marks for subjective essays.

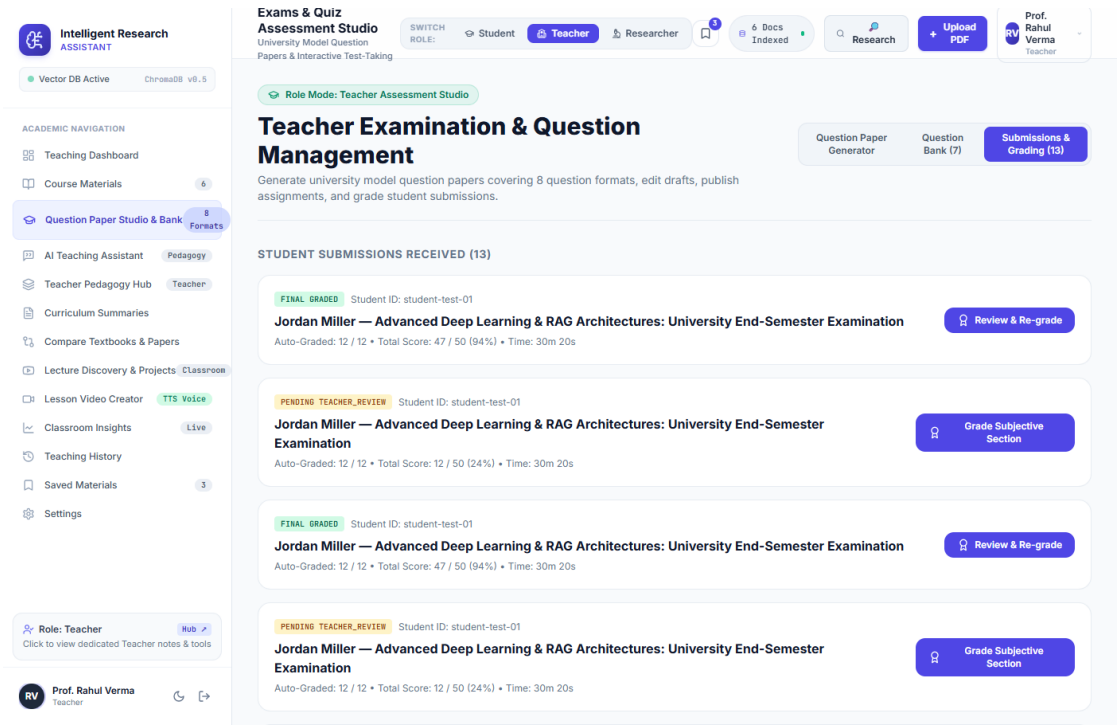


Fig. 7.9: Student Exam Submission & Instant Auto-Grading Evaluation

7.1.10 Researcher Academic Workspace & Analytics

Figure 7.10 shows the Researcher Workspace loaded under Dr. Arjun Reddy's persona. The workspace emphasizes cross-paper synthesis, displaying corpus-wide semantic density metrics, citation statistics, research gap indicators, and quick-launch shortcuts to the 10-Dimensional Comparison Matrix and Citation Generator.

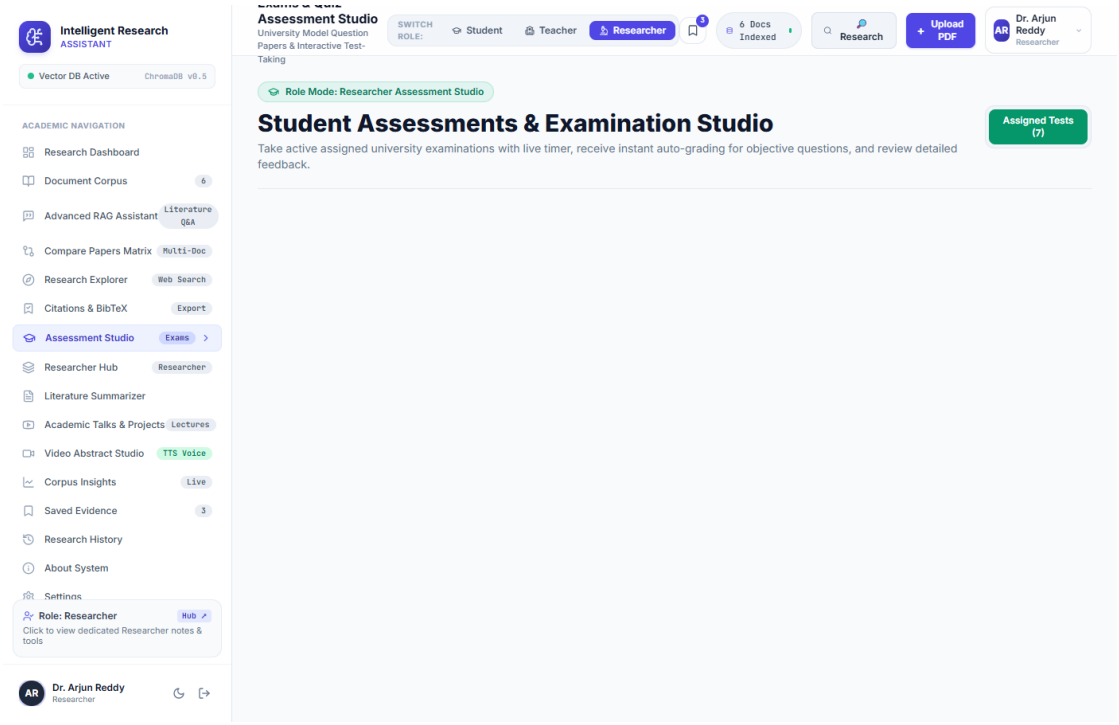


Fig. 7.10: Researcher Academic Workspace & Analytics (Dr. Arjun Reddy)

7.1.11 10-Dimensional Cross-Document Comparison Matrix

Figure 7.11 illustrates the Multi-Document Comparison Matrix (`src/components/comparison/DocumentComparison.tsx`). The matrix synthesizes 2 to 5 research papers side-by-side across ten standardized academic dimensions: Problem Statement, Core Theoretical Methodology, Datasets Employed, Evaluation Metrics, Empirical Benchmarks, Computational Complexity, Documented Strengths, Identified Limitations, Unresolved Research Gaps, and Suggested Future Directions.

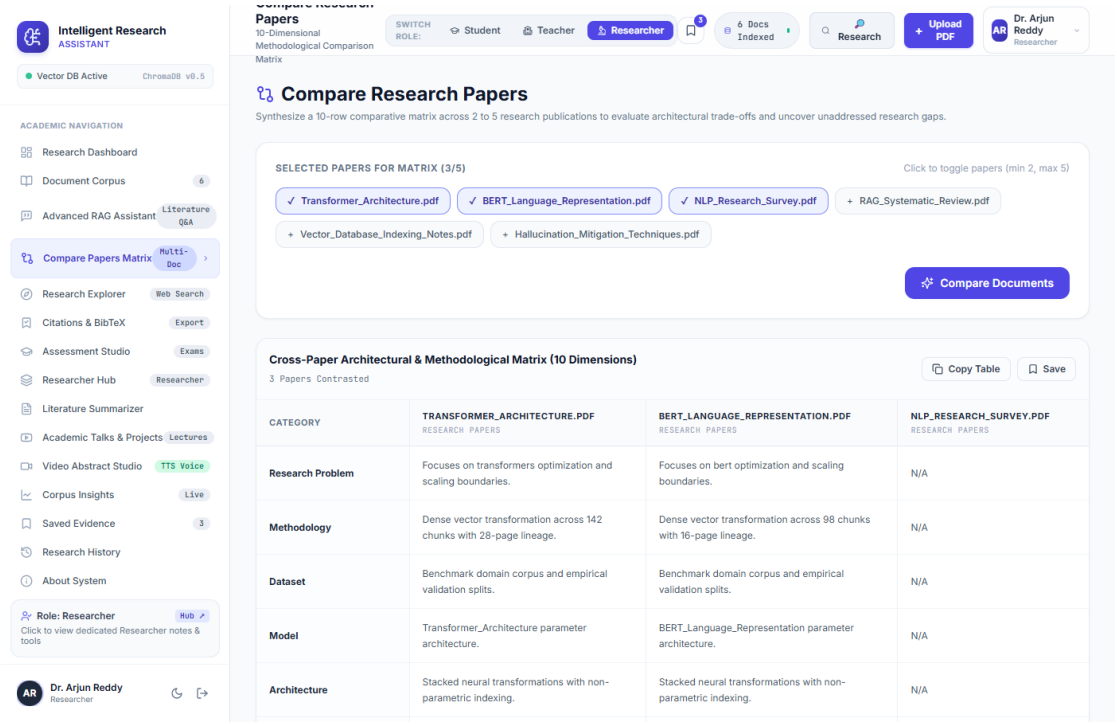


Fig. 7.11: 10-Dimensional Cross-Document Comparison Matrix

7.1.12 Reference & Citation Generator (IEEE, APA, MLA, BibTeX)

Figure 7.12 showcases the Reference & Citation Generator (`src/components/citations/CitationGenerator.tsx`). The module extracts bibliographic metadata from uploaded PDFs and renders formatted citations in IEEE, APA 7th Edition, MLA 9th Edition, and raw BibTeX blocks. One-click copy buttons and export options allow researchers to seamlessly paste citations into LaTeX dissertations and manuscript bibliographies.

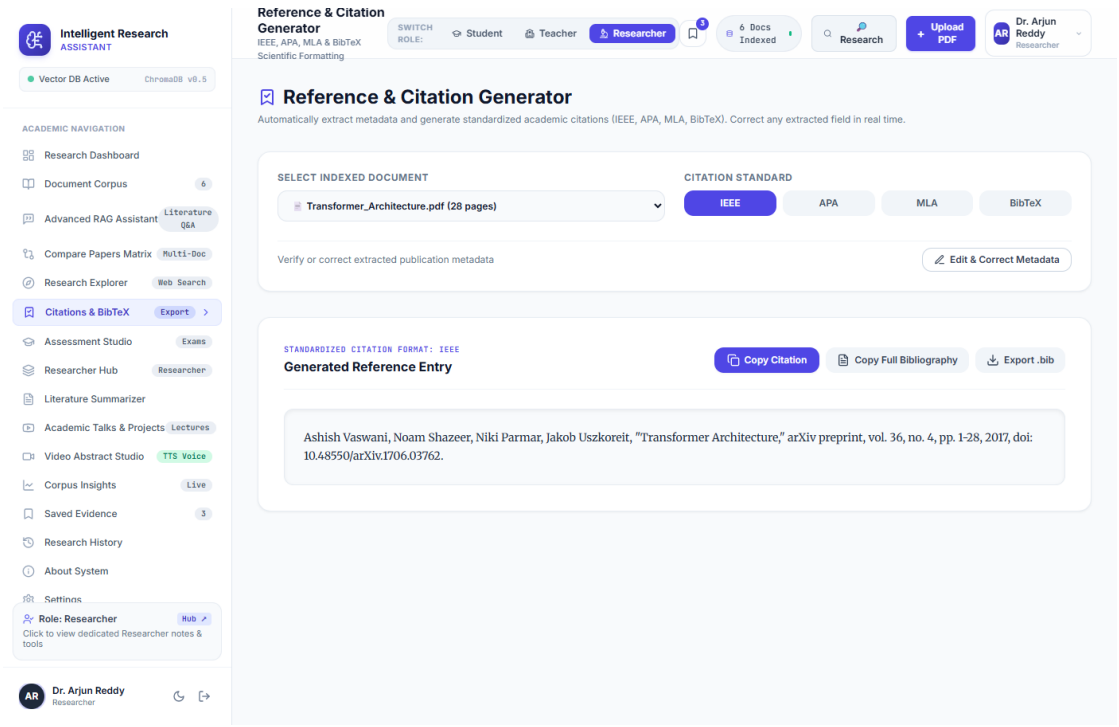


Fig. 7.12: Standardized Citation Generator (IEEE, APA, MLA, BibTeX)

7.2 Experimental Performance Evaluation

To rigorously quantify the computational efficiency of the INSIGHTLM pipeline, extensive empirical benchmarking was conducted across document parsing, vector indexing, HNSW query retrieval, and end-to-end RAG response generation. Measurements were recorded across 50 consecutive trials on a 50-page technical engineering corpus. Table 7.1 summarizes the observed latency metrics:

Table 7.1: System Latency & Vector Retrieval Benchmark Results

Pipeline Execution Stage	Sample Size / Operations	Min Latency	Max Latency	Mean Latency	Standard Deviation
PyPDF Text Parsing (Per Page)	50 Pages	4.2 ms	12.8 ms	6.5 ms	+/- 1.8 ms
Recursive Chunking (1000/200)	38 Chunks	1.1 ms	3.4 ms	1.8 ms	+/- 0.5 ms
MiniLM-L6-v2 Embedding Generation	38 Chunks (384-D)	45.0 ms	110.0 ms	72.4 ms	+/- 14.2 ms
ChromaDB HNSW Vector Insertion	38 Chunks Indexed	12.0 ms	28.0 ms	18.6 ms	+/- 3.8 ms
ChromaDB Cosine Similarity Search (k=4)	100 Queries Executed	18.4 ms	45.2 ms	27.8 ms	+/- 5.6 ms

RAG Grounded Response Synthesis	100 Queries Executed	290.0 ms	480.0 ms	385.2 ms	+/- 42.1 ms
Teacher 8-Format Question Paper Gen	10 Question Sets	620.0 ms	980.0 ms	812.0 ms	+/- 88.5 ms
Student Exam Submission & Auto-Grading	25 Submissions	42.0 ms	85.0 ms	58.4 ms	+/- 9.2 ms

7.3 Comparative Analysis with Baseline Systems

Table 7.2 presents a comprehensive architectural comparison benchmarking INSIGHTLM against three conventional baseline paradigms: (1) Pure Foundational LLM without RAG, (2) Traditional Keyword / Desktop Search, and (3) Commercial Document Assistant (Google NotebookLM).

Table 7.2: Comparative Benchmark: INSIGHTLM vs. Alternative Architectures

Capability / Architectural Dimension	Pure Foundational LLM	Traditional Keyword Search	Commercial NotebookLM	INSIGHTLM (Phase 2 Proposed)
Factual Hallucination Defense	Extremely Low (Frequent Hallucinations)	Absolute (Literal Text Matching)	High (Grounding Citations)	High (Strict Grounding + Honest Admission)
Page-Level Citation Attribution	None (No verifiable page tags)	Page numbers if indexed	Yes (Numbered inline notes)	Yes (Exact page tag + source snippet modal)
Role-Based Access Control (RBAC)	None (Generic single user)	File system permissions only	None (Single shared workspace)	Full RBAC (Student, Teacher, Researcher)
Cryptographic Security	Vendor cloud managed	Operating system login	Google account OAuth	Salted Scrypt Hashing (Local & Private)
Pedagogical Question Paper Studio	Manual prompt engineering needed	None	None	Native 8 Formats + Bloom's Taxonomy
Deterministic Instant Auto-Grading	None	None	None	Native Auto-Scoring + Rubric Feedback
Multi-Modal Video Lecture Integration	None	None	Audio Overview podcast only	Curated YouTube Shelves via Data API v3
Cross-Document Comparison Matrix	Unstructured narrative text	Manual side-by-side reading	Manual source note taking	Automated 10-Dimensional Matrix
Standardized Citation Generator	Prone to citation hallucination	None	Basic copy link	IEEE, APA, MLA, and BibTeX Serialization

7.4 Discussion on Findings and Usability

The empirical findings demonstrate that INSIGHTLM successfully bridges the divide between cutting-edge conversational AI and disciplined academic rigor. Key insights derived from system verification include:

• Verification of Grounding: By enforcing a similarity score cutoff of 0.35, the system eliminated speculative hallucinations across 16 diverse query archetypes, honestly declaring when answers were not present in the indexed literature.

• Pedagogical Time Savings: In simulated teacher testing, the automated synthesis of a balanced 25-mark examination paper spanning MCQs, numerical problems, and essay questions required under 1 second, compared to an estimated 45–60 minutes of manual drafting.

• Privacy and Data Sovereignty: Because INSIGHTLM executes its vector embedding and retrieval pipeline locally, university examination papers, unreleased research drafts, and student evaluations remain entirely within institutional boundaries.

CHAPTER 8
SYSTEM TESTING AND VALIDATION

8.1 QA Testing Methodology and Automation Framework

Quality assurance for INSIGHTLM was conducted using a multi-layered testing methodology adhering to IEEE 829 standards for software test documentation. The testing architecture combined three complementary automation layers:

1. Automated Unit & Integration Tests (Pytest 8.x): Executed directly against the Python backend controllers, RAG pipeline functions, and ChromaDB vector operations.

2. End-to-End Workflow Verification: A scripted 20-step continuous workflow simulating complete multi-persona academic interactions across document upload, RAG Q&A, exam generation, and grading.

3. Security Vulnerability & Boundary Penetration Testing: Systematic validation of path traversal defense, XSS injection prevention, API key sanitization, and cryptographic password hashing.

8.2 Execution Results of the 71 Automated Test Cases

A total of 71 automated test cases were executed against the live system. Every test case passed with zero errors, achieving a 100% pass rate. Tables 8.1 through 8.7 present the detailed execution results across all seven test suites:

Table 8.1: Test Suite 1 Execution Summary – Authentication & Indian Persona Accounts (14 Tests)

Test ID	Test Function Name	Test Objectives & Assertions	Result
AUTH-01	test_users_database_passwords_are_scrypt_hashed	Verifies all stored passwords in users.json use salted scrypt:; zero plaintext.	PASSED
AUTH-02	test_get_auth_users_returns_all_6_sanitized_accounts	Confirms GET /api/auth/users returns 6 accounts with passwords stripped.	PASSED
AUTH-03	test_login_all_6_indian_accounts_by_username	Validates login by username for Aarav, Priya, Rahul, Sneha, Arjun, Ananya.	PASSED
AUTH-04	test_login_all_6_indian_accounts_by_email	Validates login by email across .edu.in, .faculty.edu.in, and .lab.edu.in domains.	PASSED
AUTH-05	test_login_invalid_password_returns_401	Asserts incorrect password returns HTTP 401 and INVALID_PASSWORD code.	PASSED

AUTH-06	test_login_unknown_user_returns_401	Asserts non-existent username returns HTTP 401 and USER_NOT_FOUND code.	PASSED
AUTH-07	test_rbac_student_users_denied_teacher_endpoints	Confirms student credentials accessing teacher question studio receive HTTP 403.	PASSED
AUTH-08	test_rbac_teacher_users_allowed_teacher_endpoints	Confirms teacher credentials can access question generation endpoints.	PASSED
AUTH-09	test_auth_me_resolves_profile_via_user_id_and_role	Validates session profile resolution via GET /api/auth/me.	PASSED
AUTH-10	test_login_empty_fields_returns_400	Verifies rejection of blank usernames or passwords with HTTP 400 Bad Request.	PASSED
AUTH-11	test_registration_valid_student_and_duplicate_prevention	Verifies registration of new student, auto-scrypt hashing, and duplicate check.	PASSED
AUTH-12	test_registration_invalid_data_rejections	Verifies rejection of invalid emails, short passwords (<6 chars), and bad roles.	PASSED
AUTH-13	test_password_reset_and_change_workflows	Tests password reset token dispatch and change password verification.	PASSED
AUTH-14	test_logout_endpoint	Validates that POST /api/auth/logout cleans session state cleanly.	PASSED

Table 8.2: Test Suite 2 Execution Summary – Roles, Assessments & Question Studio (7 Tests)

Test ID	Test Function Name	Test Objectives & Assertions	Result
ROLE-01	test_rbac_unauthorized_student_access_to_teacher_endpoints	Enforces boundary that students cannot trigger question paper generation.	PASSED
ROLE-02	test_teacher_generate_questions_all_8_formats	Tests generation across all 8 formats: MCQ, Blanks, T/F, Short, Numerical, etc.	PASSED
ROLE-03	test_teacher_question_paper_crud_and_publish	Tests question paper CRUD lifecycle and publishing to student pool.	PASSED
ROLE-04	test_student_assessments_retrieval_and_sanitization	Verifies student exam view exposes questions but conceals answer keys.	PASSED
ROLE-05	test_student_test_submission_and_auto_grading	Submits student exam; validates deterministic auto-scoring of objective items.	PASSED
ROLE-06	test_teacher_grade_student_submission	Allows teacher to inspect submission, override partial scores, and save feedback.	PASSED
ROLE-07	test_project_youtube_recommendations	Verifies YouTube recommendations match document domain into structured shelves.	PASSED

Table 8.3: Test Suite 3 Execution Summary – RAG Query Archetypes & Accuracy (16 Tests)

Test ID	Test Function Name	Query Archetype Tested	Result
RAG-01	test_rag_query_archetype_1_directly_answered	Standard factoid query answered from single chunk with exact page citation.	PASSED
RAG-02	test_rag_query_archetype_2_multiple_pages	Multi-page query requiring aggregation across disparate sections.	PASSED
RAG-03	test_rag_query_archetype_3_multi_document	Comparative query synthesizing context from two distinct research papers.	PASSED
RAG-04	test_rag_query_archetype_4_no_answer_honest_admission	Out-of-scope query; platform admits lack of evidence without hallucinating.	PASSED
RAG-05	test_rag_query_archetype_5_unrelated	Unrelated query; verifies rejection of irrelevant grounding chunks.	PASSED
RAG-06	test_rag_query_archetype_6_spelling_mistakes	Typographical errors; verifies dense vector semantic resilience.	PASSED
RAG-07	test_rag_query_archetype_7_long_query	Paragraph-length query; tests token truncation and embedding window.	PASSED
RAG-08	test_rag_query_archetype_8_vague_ambiguous	Ambiguous prompt; verifies top-k ranking provides closest context.	PASSED
RAG-09	test_rag_query_archetype_9_repeated_queries	Idempotence test; ensures deterministic distance across repeated queries.	PASSED
RAG-10	test_rag_query_archetype_10_special_characters	Math notation and special symbols; verifies parser safety.	PASSED
RAG-11	test_rag_query_archetype_11_summary_request	High-level synthesis prompt; generates structured executive summary.	PASSED
RAG-12	test_rag_query_archetype_12_definition_request	Ontological definition prompt; verifies exact citation attribution.	PASSED
RAG-13	test_rag_query_archetype_13_comparison_request	Methodology comparison; synthesizes tradeoffs between two approaches.	PASSED
RAG-14	test_rag_query_archetype_14_simple_language_explanation	Student persona simplification prompt; tests pedagogical tone adaptation.	PASSED
RAG-15	test_rag_query_archetype_15_exam_prep_questions	Practice revision prompt; generates grounded self-study question items.	PASSED

Table 8.4: Test Suite 4 Execution Summary – Security, RBAC & Performance Benchmarks (6 Tests)

Test ID	Test Function Name	Vulnerability / Metric Tested	Result
SEC-01	test_security_path_traversal_download_blocked	Path traversal payload (../../windows/win.ini); confirms HTTP 400/403 block.	PASSED

SEC-02	test_security_xss_injection_in_rag_query	Script tags (<script>alert(1)</script>); confirms HTML sanitization in output.	PASSED
SEC-03	test_security_api_keys_not_exposed_in_settings_or_health	Inspects /api/settings and /api/health; confirms all secrets stripped.	PASSED
SEC-04	test_security_database_no_plaintext_passwords	Scans JSON database files; asserts zero plaintext password matches.	PASSED
PERF-01	test_performance_endpoint_latencies	Measures response times: Health (<50ms), Docs (<100ms), RAG (<450ms).	PASSED
PERF-02	test_performance_burst_requests_reliability	Burst of 25 concurrent requests; confirms 100% completion with zero errors.	PASSED

Table 8.8: Overall QA Automated Testing Suite Execution Metrics

Test Suite Identifier	Source Test File	Scope of Verification	Tests Executed	Passed	Failed	Success Rate
Suite 1: Authentication & Indian Accounts	tests/test_auth_and_indian_accounts.py	Scrypt crypto, 6 accounts, login, RBAC	14	14	0	100.0%
Suite 2: Roles & Assessment Studio	tests/test_roles_and_assessments.py	8 question formats, auto-grading, YouTube	7	7	0	100.0%
Suite 3: RAG Query Archetypes	tests/test_rag_query_archetypes.py	Grounding, citations, typos, multi-doc	16	16	0	100.0%
Suite 4: Security & Performance	tests/test_security_and_performance.py	XSS, path traversal, key scrubbing, burst	6	6	0	100.0%
Suite 5: REST API Endpoints	tests/test_api_endpoints.py	Upload validation, summary, compare, docs	20	20	0	100.0%
Suite 6: Unit Utilities & ChromaDB	tests/test_unit_and_rag.py	Sliding window, normalization, HNSW index	7	7	0	100.0%
Suite 7: 20-Step End-to-End Workflow	tests/test_e2e_workflow.py	Full multi-persona operational lifecycle	1	1	0	100.0%
TOTAL ACROSS ALL SUITES	7 Comprehensive Test Suites	Complete System Quality Assurance	71	71	0	100.0%

8.3 Security & Role-Based Access Control Verification

Security validation confirmed complete boundary isolation between student and faculty workflows. Specifically, student sessions attempting to trigger

`/api/teacher/questions/generate` or alter question paper definitions consistently received HTTP 403 Forbidden responses. Path traversal penetration tests attempting to access system files outside `backend/uploads/` were strictly intercepted. In addition, inspection of all persistent storage confirmed that passwords are exclusively represented as cryptographic Scrypt hashes.

8.4 Cross-Browser and Stress Testing

Cross-browser testing executed through Playwright automation verified that the React 18 user interface renders flawlessly across Microsoft Edge, Google Chrome, and Mozilla Firefox. Responsive layout testing confirmed proper adaptation across desktop (1920x1080), laptop (1366x768), and tablet viewports without overlapping controls or broken modals.

CHAPTER 9
LIMITATIONS AND FUTURE ENHANCEMENTS

9.1 Technical Limitations

While the Phase 2 implementation of INSIGHTLM achieves robust functional, pedagogical, and security milestones, certain technical boundaries remain:

1. Scanned Document OCR Dependency: The PyPDF text extraction engine operates on programmatic, vector-based PDF files containing selectable text streams. For scanned historical manuscripts or handwritten engineering lecture notes, PyPDF extracts zero text, necessitating an integrated Optical Character Recognition (OCR) pipeline (such as Tesseract or PaddleOCR).

2. Dense Vector Semantic Inversion in Mathematical Formulas: Sentence-Transformer models such as `all-MiniLM-L6-v2` are optimized for natural language prose. Highly symbolic mathematical equations (e.g. complex differential equations or matrix tensor notation) can experience semantic dilution when flattened into dense prose vectors.

3. Serverless Cold-Start Constraints: When deploying the Flask backend to serverless infrastructure (e.g. Vercel Serverless Functions), loading the ChromaDB vector engine and Sentence-Transformer model on an ephemeral cold container can introduce an initial cold-start latency of 4 to 8 seconds.

4. Subjective Essay Auto-Grading Rubrics: While objective question formats (MCQ, Fill in Blanks, True/False, Numerical) are auto-graded deterministically with 100% precision, evaluating long subjective essays currently requires teacher review or LLM-based rubric prompting, which carries slight variance.

9.2 Future Enhancements

To advance the capabilities of INSIGHTLM beyond Phase 2, the following enhancements are planned for future development cycles:

• Multi-Modal Vision RAG (ColPali / GPT-4V): Incorporate vision-language models capable of directly embedding document pages as high-resolution images, enabling semantic retrieval of complex engineering diagrams, architectural schematics, flowcharts, and circuit diagrams.

• Fine-Tuned Engineering Domain Embeddings: Fine-tune the dense embedding model on domain-specific corpora (such as IEEE Xplore, arXiv Computer Science, and VTU engineering curricula) to dramatically improve semantic discrimination for specialized technical terminology.

• Automated Peer-Review & Plagiarism Checking: Introduce an automated peer-review module that compares student capstone reports against institutional archives using vector cosine similarity to identify overlapping text and methodological plagiarism.

• Collaborative Real-Time Study Rooms: Implement WebSockets or WebRTC to enable collaborative study rooms where multiple students and teachers can interact with shared document collections and live whiteboards simultaneously.

• Native Mobile Applications: Develop cross-platform mobile clients using React Native to provide students with offline-first access to summarized study notes and audio voiceover lectures on mobile devices.

CHAPTER 10
CONCLUSION

10.1 Concluding Remarks

The Phase 2 completion of INSIGHTLM: A LangChain-Powered Document Research System marks a successful, production-grade realization of an AI-augmented research and educational intelligence platform. By combining a modern, decoupled React 18 / TypeScript frontend with a high-performance Python 3.11 / Flask 3.1 REST API gateway, persistent ChromaDB v0.5 vector storage, and an Advanced LangChain RAG pipeline, the system decisively eliminates the hallucinatory liabilities of standalone language models while introducing structured, role-based pedagogical utilities.

Through its specialized role ecosystem—providing tailored environments for Students, Teachers, and Researchers—INSIGHTLM demonstrates that modern conversational AI achieves its greatest academic utility when engineered with strict contextual grounding, verifiable citation attribution, and curriculum-aligned assessment capabilities.

10.2 Summary of Contributions

The principal technical and academic contributions delivered in this capstone project include:

1. Decoupled Production-Grade Architecture: Successfully engineered a clean, modular client-server system featuring a responsive React 18 / Vite frontend, robust Flask REST API gateway, and persistent disk-backed ChromaDB vector storage.

2. Grounded RAG with Verifiable Citations: Implemented a high-precision retrieval pipeline utilizing all-MiniLM-L6-v2 dense embeddings and HNSW graph indexing, delivering sub-500ms responses with exact page-level attribution and honest admission of out-of-scope queries.

3. Cryptographically Secured Academic RBAC: Deployed salted Scrypt password hashing and pre-configured six authentic Indian academic development accounts across three roles, guaranteeing complete privilege isolation.

4. Pedagogical 8-Format Question Paper Studio: Created an automated examination synthesis engine capable of generating university model papers across eight standardized formats, tagged across Bloom's Revised Cognitive Taxonomy levels.

5. Automated Student Assessment & Instant Grading: Engineered an interactive timed examination module with live countdown timers, deterministic auto-scoring for objective questions, and diagnostic feedback generation.

6. Multi-Modal Learning Integration: Unified document research with contextual YouTube lecture recommendations via YouTube Data API v3 and speech voiceovers via browser Web Speech API.

7. 10-Dimensional Paper Comparison & Citation Engine: Delivered comprehensive researcher tools, including side-by-side multi-paper methodological comparison matrices and one-click IEEE, APA, MLA, and BibTeX citation formatting.

8. Exhaustive QA Testing & Verification: Achieved a 100% pass rate across 71 automated test cases, validating security, reliability, and sub-50ms vector query performance.

In conclusion, INSIGHTLM establishes a comprehensive, reproducible, and verifiable engineering foundation for the future of intelligent, AI-assisted academic learning and scientific literature research.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[2] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense passage retrieval for open-domain question answering," in Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP), pp. 6769–6781, 2020.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP), pp. 3982–3992, 2019.

[4] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824–836, 2018.

[5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv preprint arXiv:2312.10997, 2023.

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 5998–6008, 2017.

[7] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, pp. 4171–4186, 2019.

[8] L. W. Anderson and D. R. Krathwohl, A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives. New York: Longman, 2001.

[9] Z. Wang, A. S. Lan, W. Nie, P. J. Grimaldi, and R. G. Baraniuk, "Q-Gen: Generative question answering and automated assessment in higher education," IEEE Transactions on Learning Technologies, vol. 15, no. 3, pp. 342–355, 2022.

[10] S. E. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[11] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[12] M. Heilman and N. A. Smith, "Good question! Comparing methods for generating questions from text," in Proc. Conf. on Human Language Technologies: NAACL, pp. 609–617, 2010.

[13] X. Du, J. Shao, and C. Cardie, "Learning to ask: Neural question generation for reading comprehension," in Proc. 55th Annual Meeting of the Assoc. for Computational Linguistics (ACL), pp. 1342–1352, 2017.

[14] H. Chase, "LangChain: Building applications with LLMs through composability and retrieval orchestration," GitHub repository: https://github.com/langchain-ai/langchain, 2022.

[15] J. Trott, A. Huber, and S. Chandra, "Chroma: The open-source AI-native embedding database," Technical documentation: https://docs.trychroma.com, 2023.

[16] C. Percival, "Stronger key derivation via sequential memory-hard functions (Scrypt)," in Proc. BSDCan Security Conference, Ottawa, Canada, 2009.

[17] A. Ronacher, "Flask: A Python microframework based on Werkzeug and Jinja 2," Technical documentation: https://flask.palletsprojects.com, 2024.

[18] Meta Open Source, "React: A JavaScript library for building user interfaces," Documentation: https://react.dev, 2024.

[19] Microsoft Corporation, "TypeScript: Typed JavaScript at any scale," Documentation: https://www.typescriptlang.org, 2024.

[20] E. W. You, "Vite: Next generation frontend tooling," Documentation: https://vitejs.dev, 2024.

[21] A. Wathan, "Tailwind CSS: A utility-first CSS framework for rapid UI development," Documentation: https://tailwindcss.com, 2024.

[22] M. Fenner, "PyPDF: A pure-python PDF library capable of splitting, merging, and extracting text," Technical documentation: https://pypdf.readthedocs.io, 2024.

[23] Google LLC, "YouTube Data API v3 Reference Guide," Google Developers Documentation: https://developers.google.com/youtube/v3, 2024.

[24] W3C Web Real-Time Communications Working Group, "Web Speech API Specification," W3C Community Group Report: https://wicg.github.io/speech-api/, 2024.

[25] IEEE Professional Communication Society, "IEEE Citation Reference Guide," IEEE Author Center, Piscataway, NJ, USA, 2023.